IMPERIAL

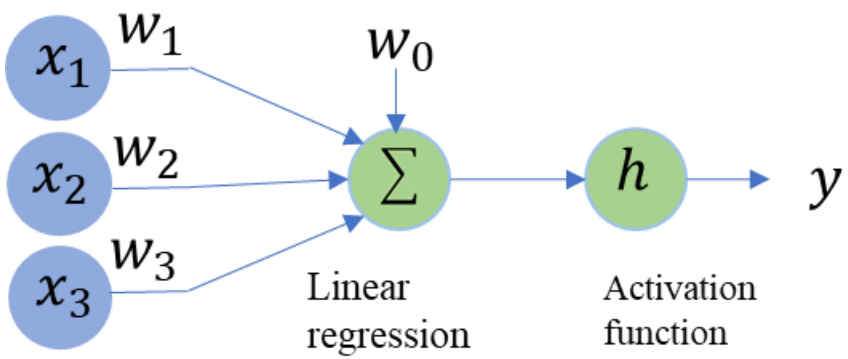
# Department of Chemical Engineering

# Machine Learning for Chemical Engineering

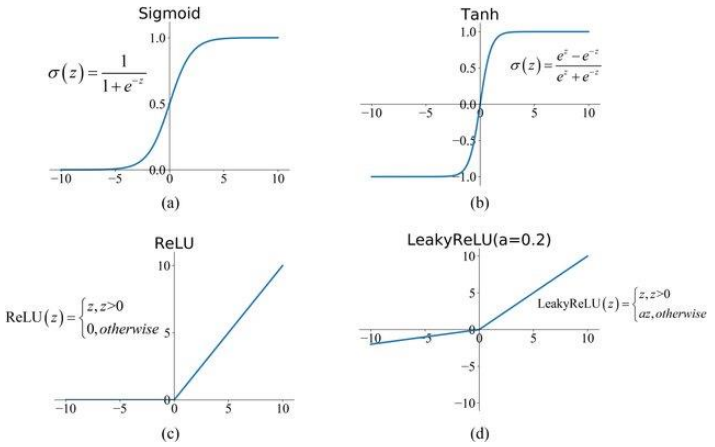
## Lecture 2b – Optimisation Algorithms for Deep Learning

# Recall

A **linear regression** is contained in every neuron in a neural network.

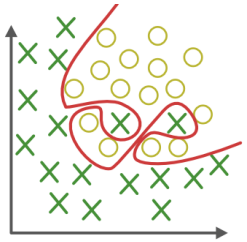


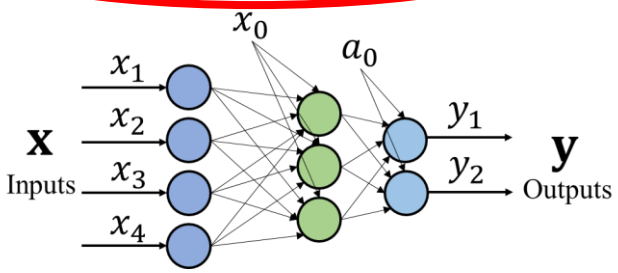

Neural networks use **activation functions** as nonlinear transformations of their inputs.

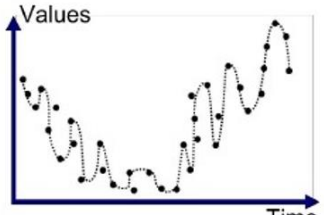



NNs are **overparametrised** functions which are **'trained' through optimisation**.

$$\min_{W^{(1)},\dots,W^{(n_{hl})}} \mathcal{L}\left(W^{(1)},\dots,W^{(n_{hl})};\mathbf{X},\mathbf{Y}\right)$$

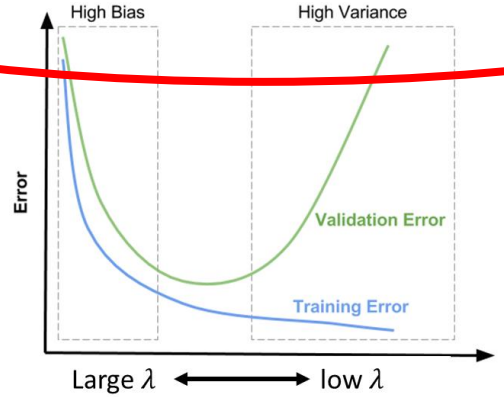




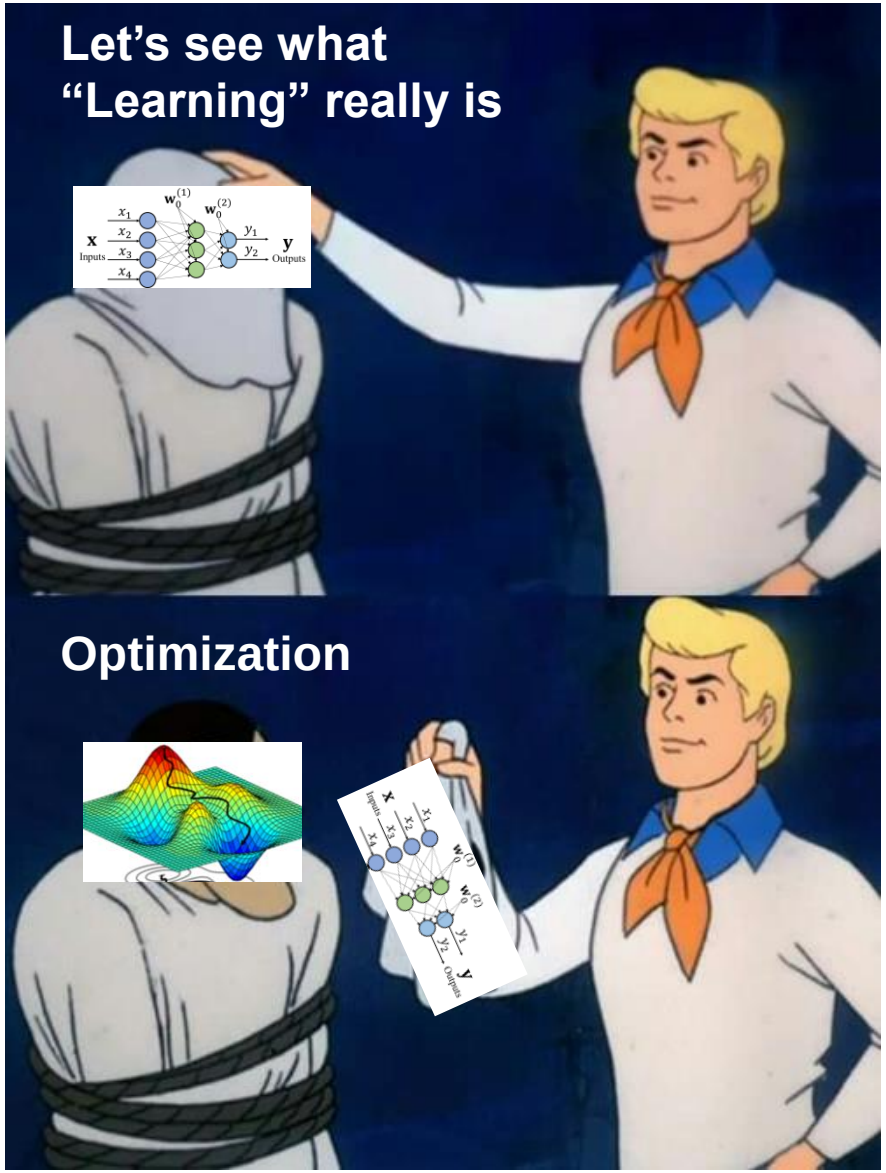




To address overfitting we can use cross-validation

$$\min_{W^{(1)},\dots,W^{(n_{hl})}} \mathcal{L}\left(W^{(1)},\dots,W^{(n_{hl})};\mathbf{X},\mathbf{Y}\right) + \lambda \sum_{i=1}^{n_w} w_i^2$$



# What learning really is? (in NNs and ML)



# What learning really is? (in NNs and ML)

## Supervised Learning

$$\boldsymbol{\theta}_{\mathrm{ML}} = \arg\max_{\boldsymbol{\theta}} P(\boldsymbol{Y} \mid \boldsymbol{X}; \boldsymbol{\theta}).$$

$$\mathrm{MSE}_{\mathrm{test}} = \frac{1}{m} \sum_i (\hat{\boldsymbol{y}}^{(\mathrm{test})} - \boldsymbol{y}^{(\mathrm{test})})_i^2.$$

## Reinforcement Learning

**Policy Gradients**

$$\theta^\star = \arg\max_\theta \underbrace{E_{\tau \sim p_\theta(\tau)} \left[ \sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]}_{J(\theta)}$$

**Q-Learning**

$$\mathbf{y}_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\phi(\mathbf{s}'_i, \mathbf{a}'_i)$$

$$\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(\mathbf{s}_i, \mathbf{a}_i)(Q_\phi(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{y}_i)$$

## Unsupervised Learning

**K-means**

$$\arg\min_{\mathbf{S}} \sum_{i=1}^{k} \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2 = \arg\min_{\mathbf{S}} \sum_{i=1}^{k} |S_i| \operatorname{Var} S_i$$

**Community Detection**

$$\begin{aligned} Q_{max} &= \max_{\mathcal{P}} \left\{ \sum_{c=1}^{n_c} \left[ \frac{l_c}{m} - \left( \frac{d_c}{2m} \right)^2 \right] \right\} \\ &= \frac{1}{m} \max_{\mathcal{P}} \left\{ \sum_{c=1}^{n_c} \left[ l_c - \mathrm{Ex}(l_c) \right] \right\} \\ &= -\frac{1}{m} \min_{\mathcal{P}} \left\{ -\sum_{c=1}^{n_c} \left[ l_c - \mathrm{Ex}(l_c) \right] \right\} \end{aligned}$$

# Gradient Descent

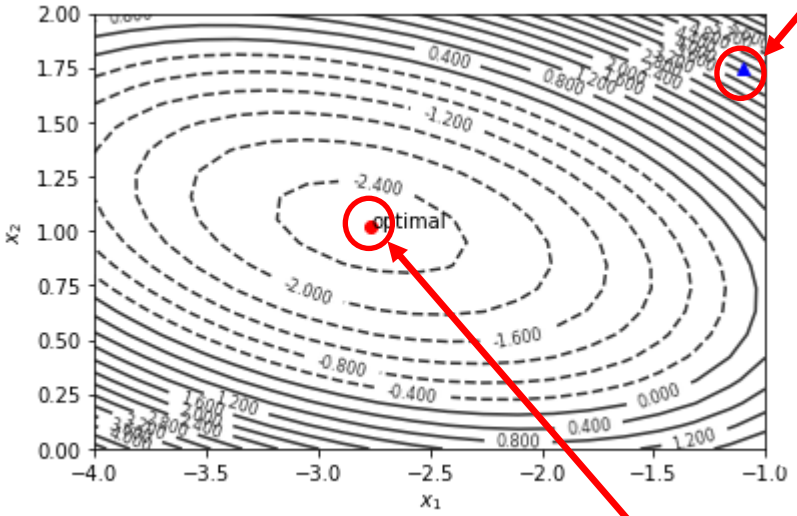
# Continuous Unconstrained Optimization – Problem statement

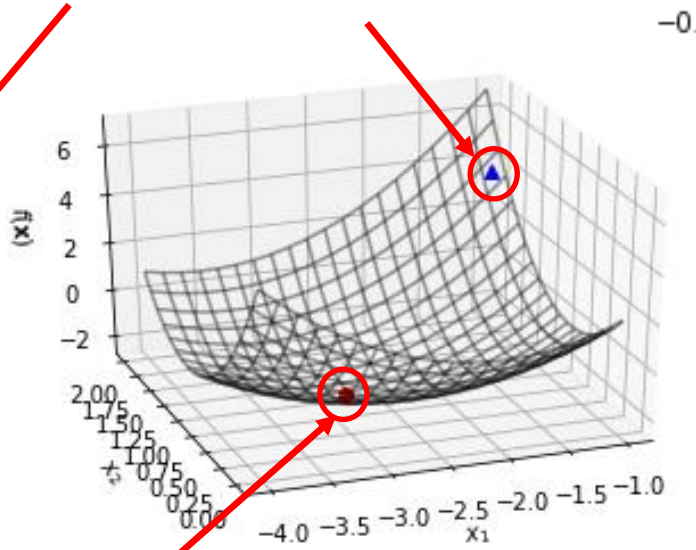
## Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

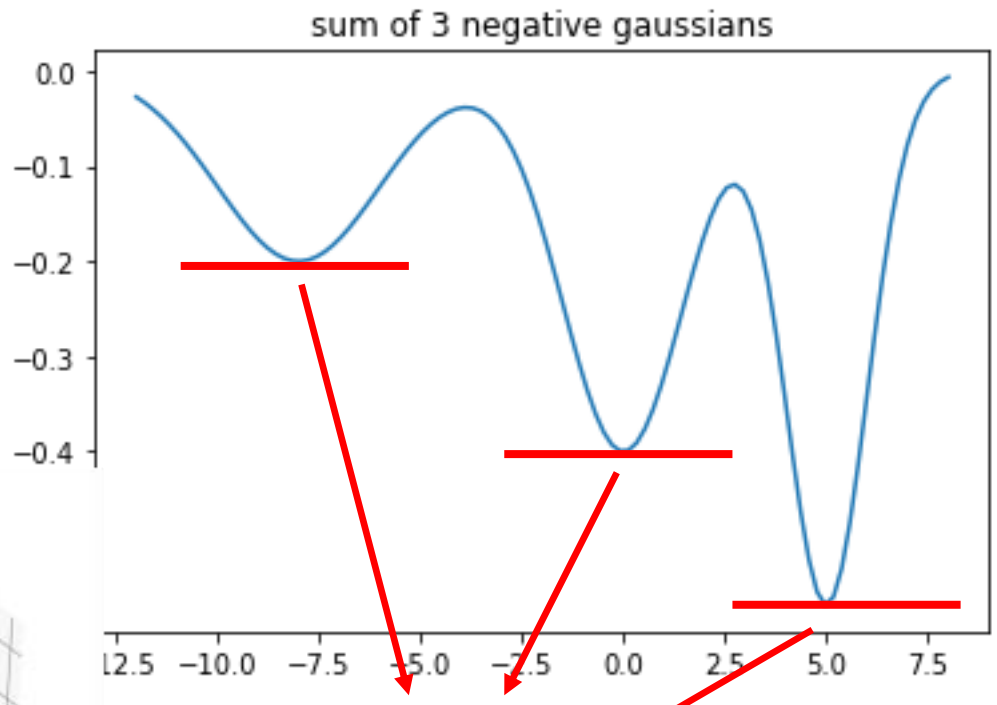
$$f: \mathbb{R}^{n_x} \rightarrow \mathbb{R}$$

starting point




optimum




$$\nabla_{\mathbf{x}} f = \mathbf{0}$$

- The **gradient** is the **slope**
- The slope is **zero** at the **optimum**

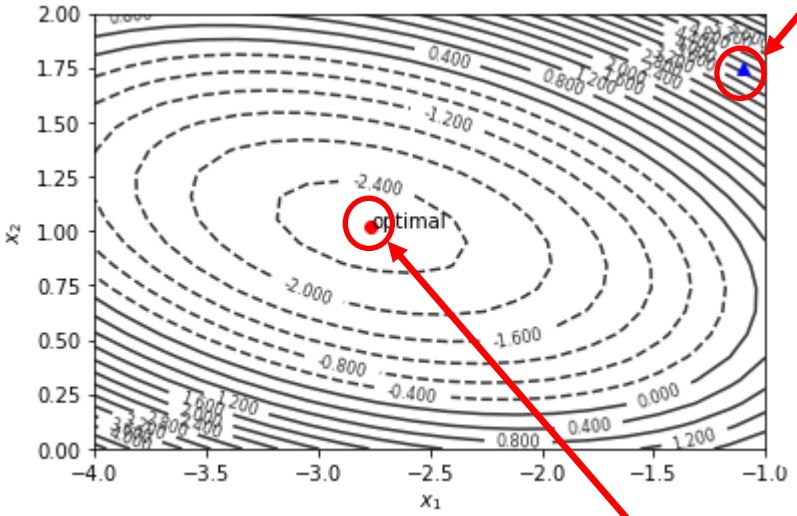
# Continuous Unconstrained Optimization – Problem statement

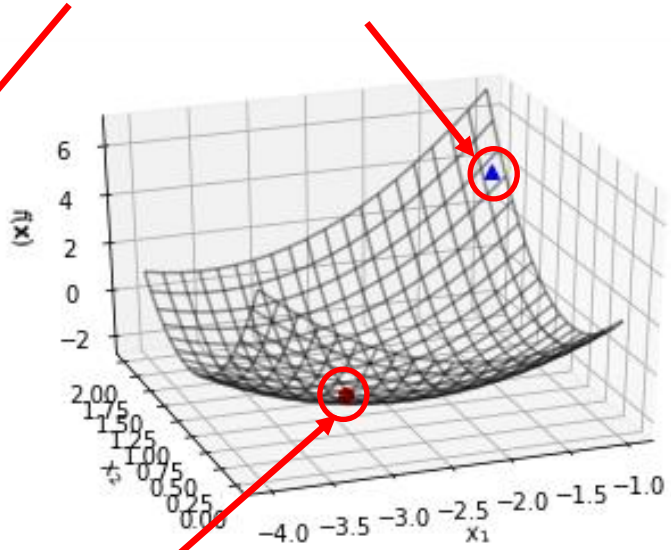
## Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

$$f: \mathbb{R}^{n_x} \rightarrow \mathbb{R}$$

starting point




optimum

## Gradient Descent

Going in the direction of the most rapid function decrease

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \ \nabla_x f(\mathbf{x}_k)$$

$$\nabla_x f(\mathbf{x}_k) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}_k) \\ \vdots \\ \frac{\partial f}{\partial x_{n_x}}(\mathbf{x}_k) \end{bmatrix}$$

**Example:**

$$f(\mathbf{x}) = (x_1 - 2)^2 + x_1 x_2$$

$$\nabla_x f(\mathbf{x}_k) = \begin{bmatrix} 2(x_1 - 2) + x_2 \\ x_1 \end{bmatrix}$$

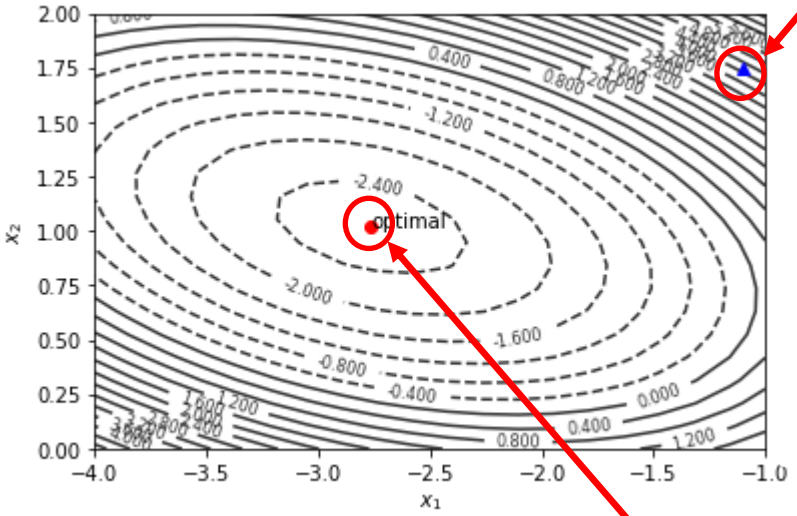
# Continuous Unconstrained Optimization – Problem statement

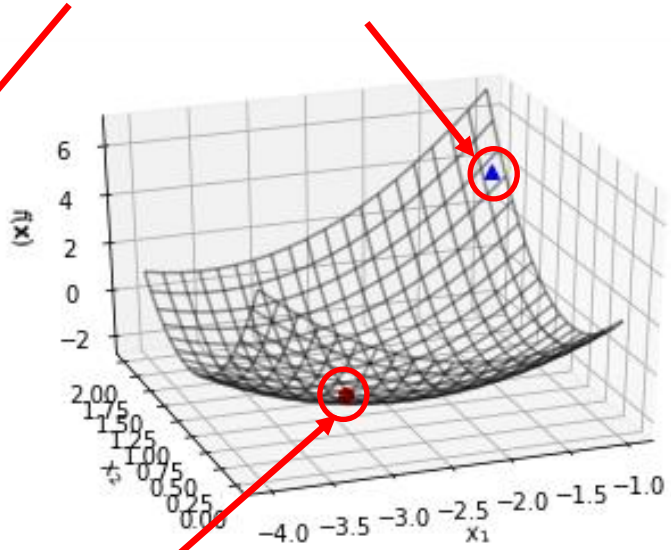
## Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

$$f: \mathbb{R}^{n_x} \to \mathbb{R}$$

starting point




optimum

## Gradient Descent

Going in the direction of the most rapid function decrease

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \ \nabla_x f(\mathbf{x}_k)$$

$$\nabla_x f(\mathbf{x}_k) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}_k) \\ \vdots \\ \frac{\partial f}{\partial x_{n_x}}(\mathbf{x}_k) \end{bmatrix}$$

**Example:**

$$f(\mathbf{x}) = (x_1 - 2)^2 + x_1 x_2 + x_2 \log x_3$$

$$\nabla_x f(\mathbf{x}_k) = \begin{bmatrix} 2(x_1 - 2) + x_2 \\ x_1 + \log x_3 \\ x_2 / x_3 \end{bmatrix}$$

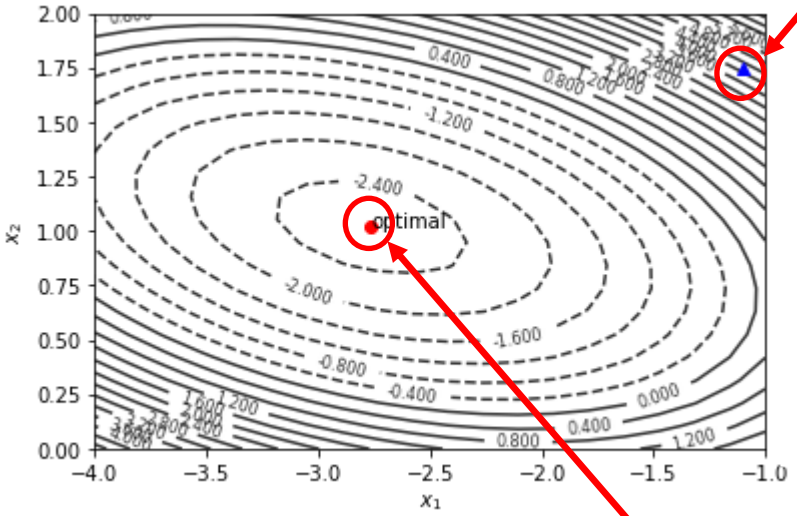
# Continuous Unconstrained Optimization – Problem statement

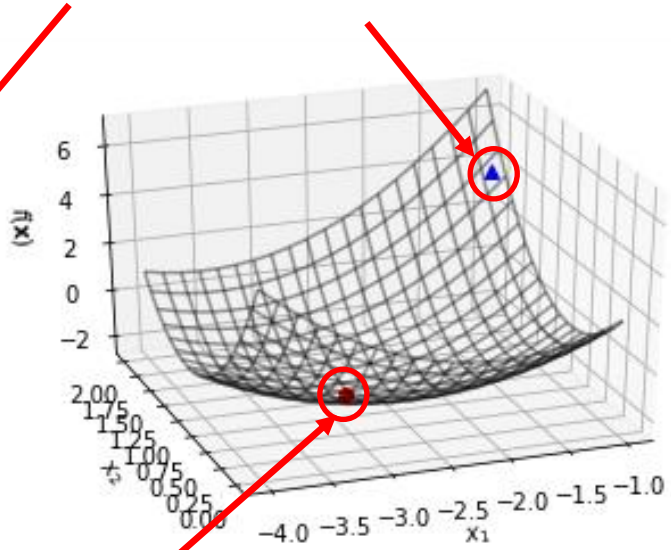
## Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

$$f: \mathbb{R}^{n_x} \rightarrow \mathbb{R}$$

starting point




optimum

## Gradient Descent

Going in the direction of the most rapid function decrease

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \boxed{\nabla_x f(\mathbf{x}_k)}$$

Step size $\approx 10^{-3} - 10^{-5}$

Fastest direction of function increase

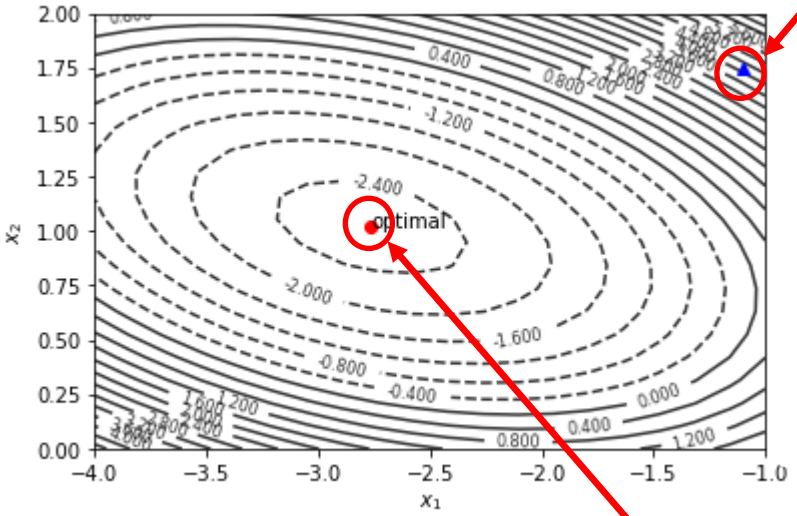
# Continuous Unconstrained Optimization – Problem statement

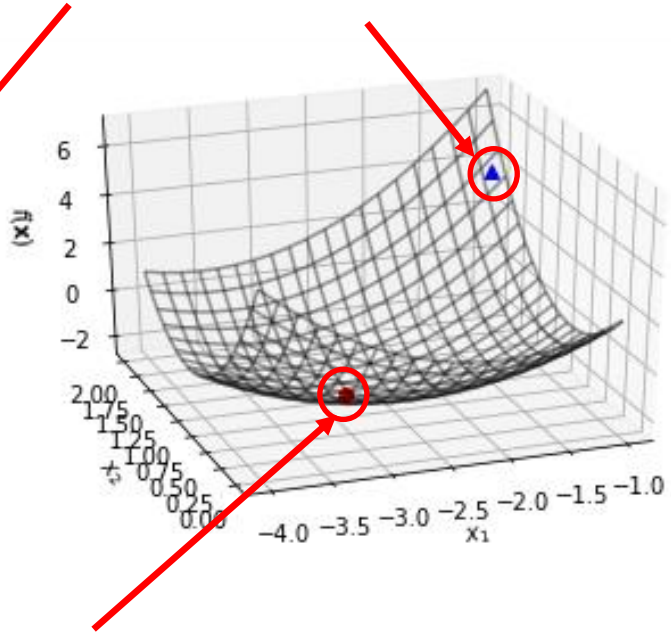
## Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

$$f: \mathbb{R}^{n_x} \to \mathbb{R}$$

starting point




optimum

## Gradient Descent

Going in the direction of the most rapid function decrease

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \ \nabla_x f(\mathbf{x}_k)$$

**Algorithm: Gradient Descent**

1. **Start** $\mathbf{x}_k := \mathbf{x}_0$
2. **Until** $\|\nabla_x f(\mathbf{x}_k)\| \leq \varepsilon$

   $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \ \nabla_x f(\mathbf{x}_k)$

   **end**

with $\varepsilon \approx 10^{-5}$

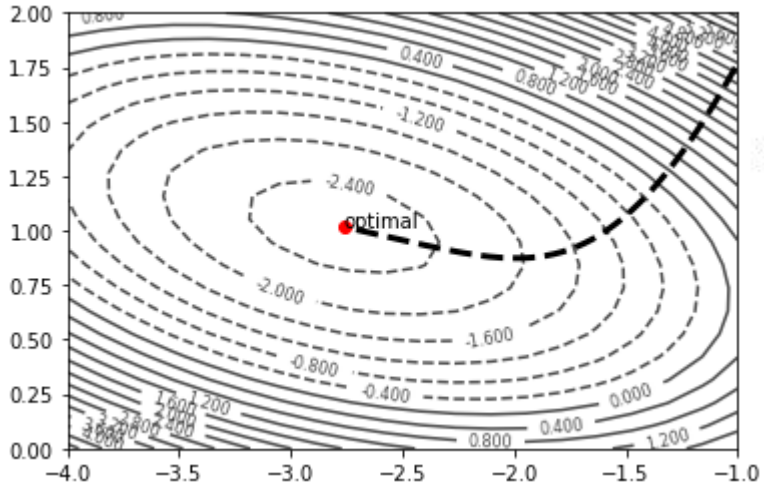
# Continuous Unconstrained Optimization – Problem statement

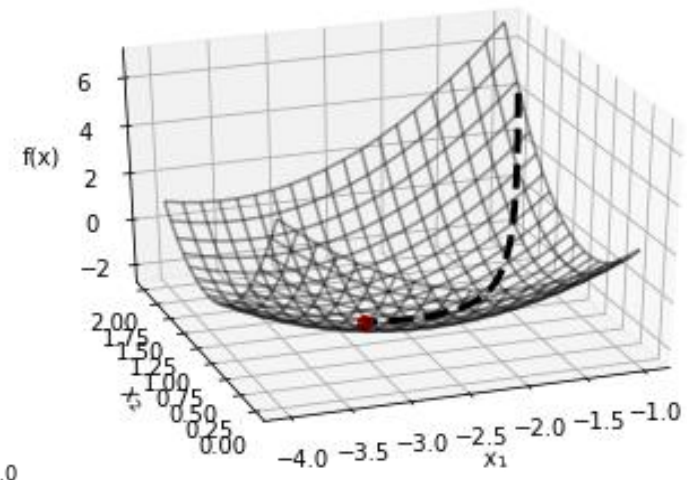
## Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

$$f: \mathbb{R}^{n_x} \to \mathbb{R}$$

Number of steps ~ 60k !




## Gradient Descent

Going in the direction of the most rapid function decrease

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \ \nabla_x f(\mathbf{x}_k)$$

**Algorithm: Gradient Descent**

1. **Start** $\mathbf{x}_k := \mathbf{x}_0$
2. **Until** $\|\nabla_x f(\mathbf{x}_k)\| \leq \varepsilon$

   $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \ \nabla_x f(\mathbf{x}_k)$

   **end**

with $\varepsilon \approx 10^{-5}$

Computer science literature: <u>greedy algorithm</u>

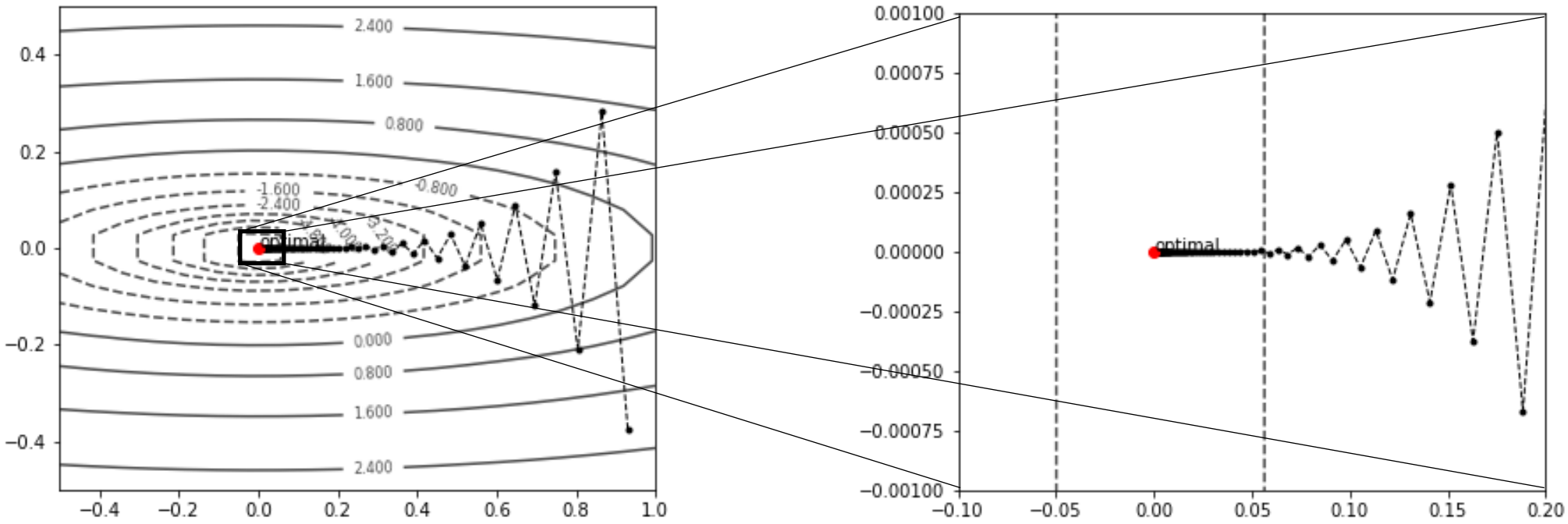
<u>Terrible performance!</u>

# Continuous Unconstrained Optimization – Gradient Descent

## Problem statement

Scaling matters in gradient descent!

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x}) := (x_1)^2 + (5x_2)^2$$

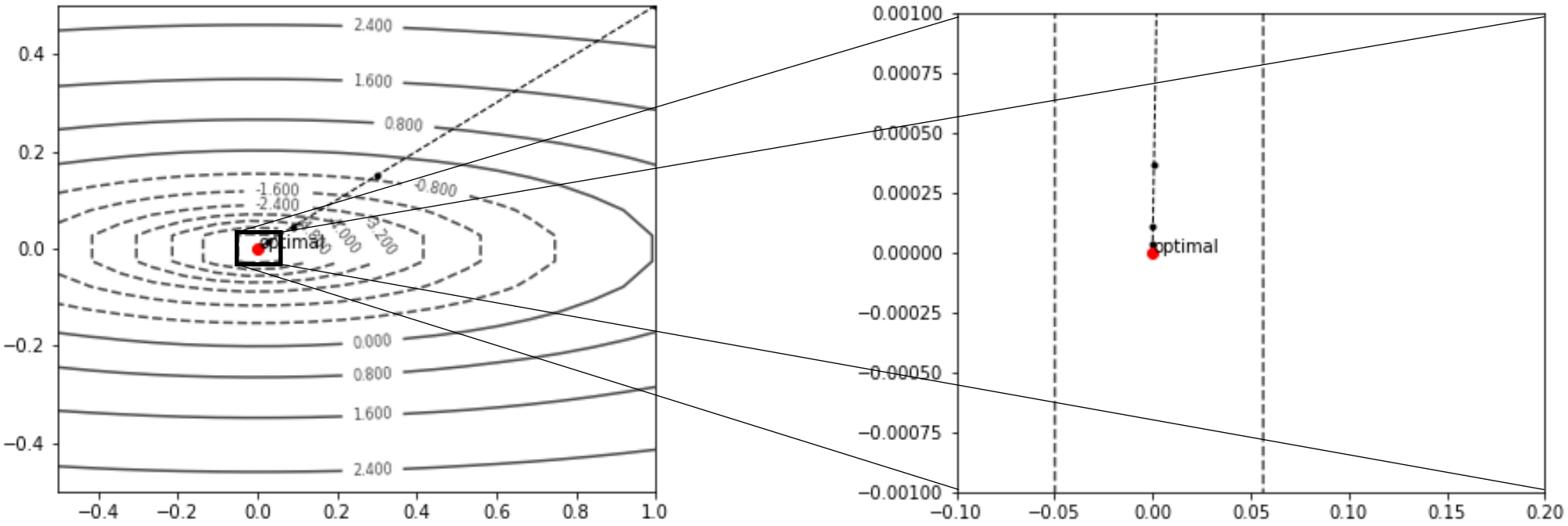


# Continuous Unconstrained Optimization – Gradient Descent

## Problem statement

Scaling matters in gradient descent!

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x}) := (x_1)^2 + (1x_2)^2$$

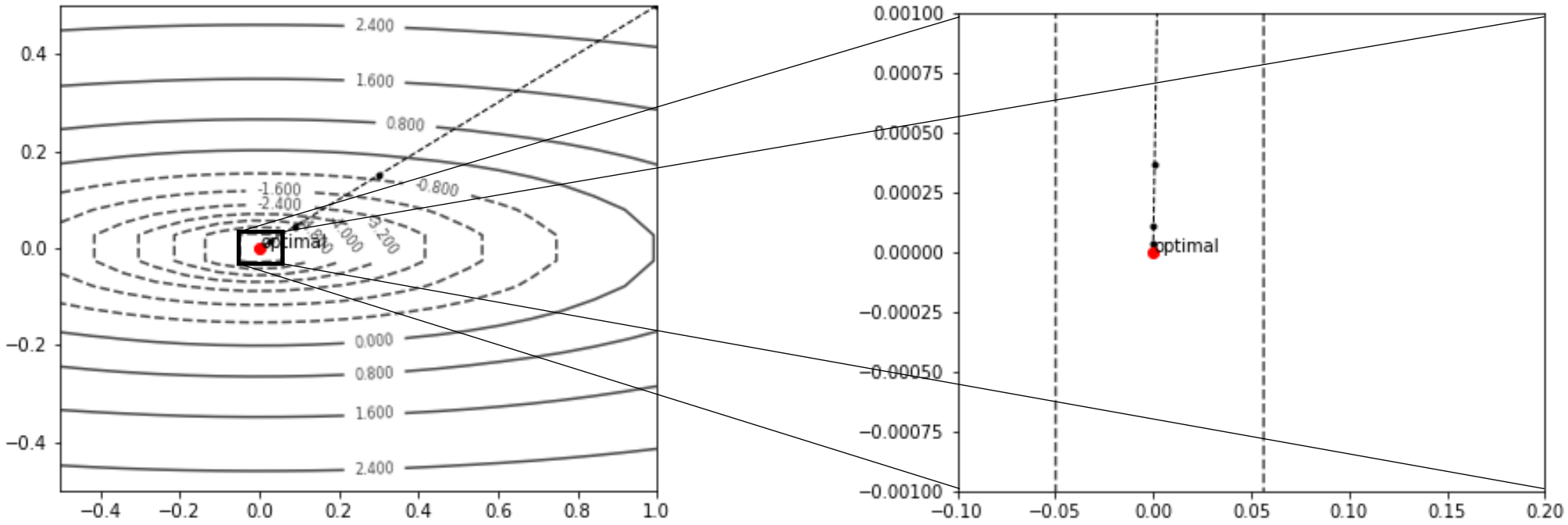


# Continuous Unconstrained Optimization – Gradient Descent

## Problem statement

- Many times faster convergence!
- More a problem than a solution.

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x}) := (x_1)^2 + (1x_2)^2$$



# Continuous Unconstrained Optimization – Gradient Descent

**Gradient Descent key takeaways:**

- Takes steps in the direction on the negative gradient.
- First order method (only uses gradients)
- It is heavily affected by scaling and ill conditioned functions.
- The error decreases linearly, for example:
  - $\varepsilon_{k+1} = \varepsilon_k(0.9)$
  - $\varepsilon(k) = (0.5)^k$
- Terrible for high precision
- Not used in optimization practice. **Except in ML training!**

# Newton’s Method (Optimization)

# Continuous Unconstrained Optimization – Newton's Method

**Newton step**

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \nabla^2 f(\mathbf{x}_k)^{-1} \nabla f(\mathbf{x}_k)$$

Gradient Descent

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \quad \nabla f(\mathbf{x}_k)$$

**Step size:**

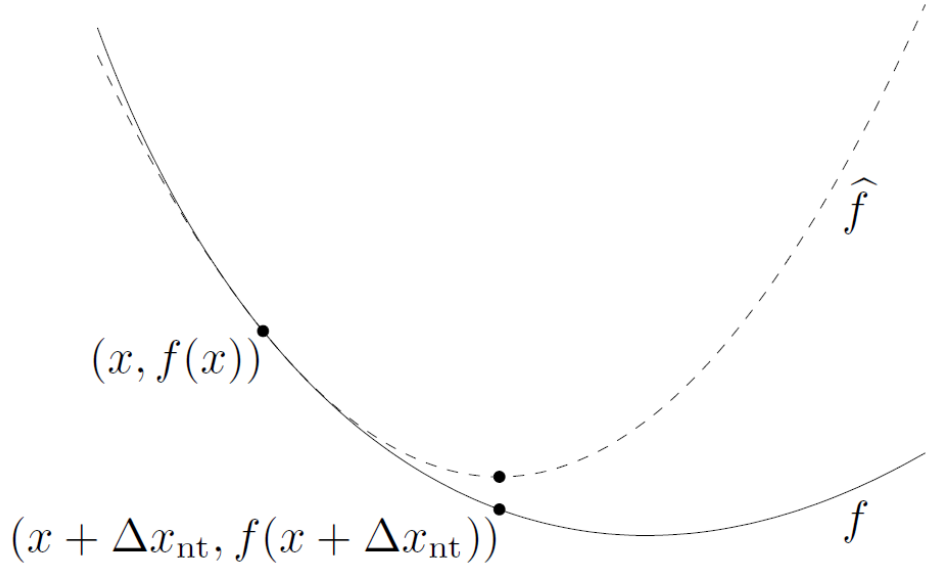
Takes into account the curvature by the Hessian (matrix of second derivatives)

$$\nabla^2 f(\mathbf{x}_k) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 x_{n_x}} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_{n_x} x_1} & \cdots & \frac{\partial^2 f}{\partial x_{n_x}^2} \end{bmatrix}$$

**Newton step interpretation 1**

Minimizer of second-order approximation

$$\hat{f}(\mathbf{x}_k + \Delta\mathbf{x}) = f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^T \Delta\mathbf{x} + \frac{1}{2} \Delta\mathbf{x}^T \nabla^2 f(\boldsymbol{x}_k) \Delta\mathbf{x}$$




* Image from Convex Optimization by S.P. Boyd, L. Vandenberghe

# Continuous Unconstrained Optimization – Newton's Method

**Newton step**

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \nabla^2 f(\mathbf{x}_k)^{-1} \nabla f(\mathbf{x}_k)$$

Gradient Descent

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \quad \nabla f(\mathbf{x}_k)$$

**Step size:**

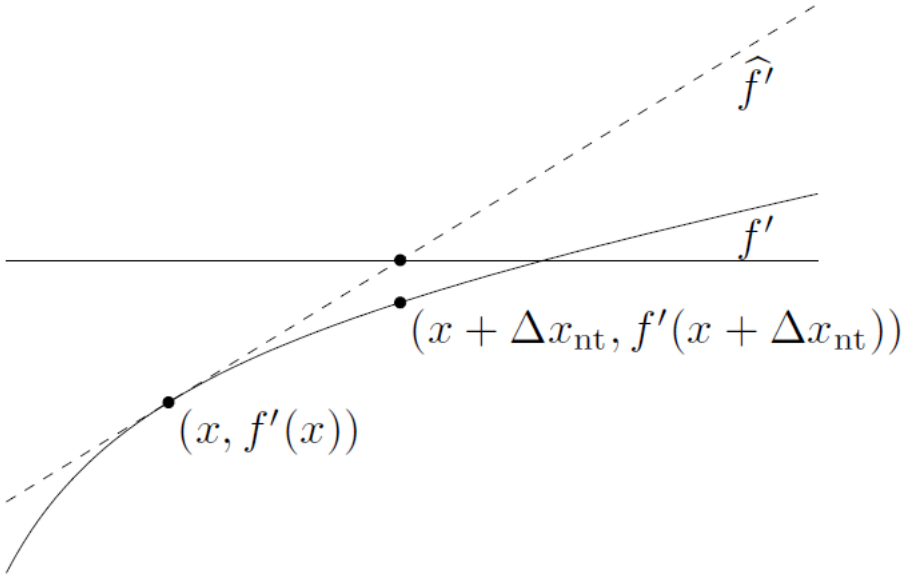
Takes into account the curvature by the Hessian (matrix of second derivatives)

$$\nabla^2 f(\mathbf{x}_k) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 x_{n_x}} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_{n_x} x_1} & \cdots & \frac{\partial^2 f}{\partial x_{n_x}^2} \end{bmatrix}$$

**Newton step interpretation 2** $\quad \nabla f(\mathbf{x}) = \mathbf{0}$

Solution of linearized optimality condition

$$\nabla f(\mathbf{x}_k + \Delta\mathbf{x}) \approx \nabla f(\mathbf{x}_k) + \nabla^2 f(\mathbf{x}_k)\Delta\mathbf{x} = 0$$




* Image from Convex Optimization by S.P. Boyd, L. Vandenberghe

# Continuous Unconstrained Optimization – Newton's Method

**Newton step**

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \nabla^2 f(\mathbf{x}_k)^{-\mathbf{1}}\, \nabla f(\mathbf{x}_k)$$

Gradient Descent

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \quad \nabla f(\mathbf{x}_k)$$

**Step size:**

Takes into account the curvature by the Hessian (matrix of second derivatives)

$$\nabla^2 f(\mathbf{x}_k) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 x_{n_x}} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_{n_x} x_1} & \cdots & \frac{\partial^2 f}{\partial x_{n_x}^2} \end{bmatrix}$$

**Newton step interpretation 2** $\quad \nabla f(\mathbf{x}) = \mathbf{0}$

Solution of linearized optimality condition

$$\nabla f(\mathbf{x}_k + \Delta\mathbf{x}) \approx \nabla f(\mathbf{x}_k) + \nabla^2 f(\mathbf{x}_k)\Delta\mathbf{x} = 0$$

**Algorithm: Newton's Method**

1. **Start** $\mathbf{x}_k := \mathbf{x}_0$
2. **Until** $\|\boldsymbol{\nabla}_{\boldsymbol{x}} f(\mathbf{x}_k)\| \leq \varepsilon$

   $\mathbf{x}_{k+1} = \mathbf{x}_k - \nabla^2 f(\mathbf{x}_k)^{-\mathbf{1}} \nabla_{\boldsymbol{x}} f(\mathbf{x}_k)$

**end**

# Continuous Unconstrained Optimization – Newton's Method

**Newton step**

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \nabla^2 f(\mathbf{x}_k)^{-1}\, \nabla f(\mathbf{x}_k)$$

Gradient Descent

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \quad \nabla f(\mathbf{x}_k)$$

**Step size:**

Takes into account the curvature by the Hessian (matrix of second derivatives)

$$\nabla^2 f(\mathbf{x}_k) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 x_{n_x}} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_{n_x} x_1} & \cdots & \frac{\partial^2 f}{\partial x_{n_x}^2} \end{bmatrix}$$

**Newton step interpretation 2** $\quad \nabla f(\mathbf{x}) = \mathbf{0}$

Solution of linearized optimality condition

$$\nabla f(\mathbf{x}_k + \Delta\mathbf{x}) \approx \nabla f(\mathbf{x}_k) + \nabla^2 f(\mathbf{x}_k)\Delta\mathbf{x} = 0$$

Newton decrement

**Algorithm: Newton's Method**

1. **Start** $\mathbf{x}_k := \mathbf{x}_0$
2. **Until** $\boxed{\nabla f(\mathbf{x}_k)^T \nabla^2 f(\mathbf{x}_k)^{-1} \nabla_x f(\mathbf{x}_k)} \leq \varepsilon$

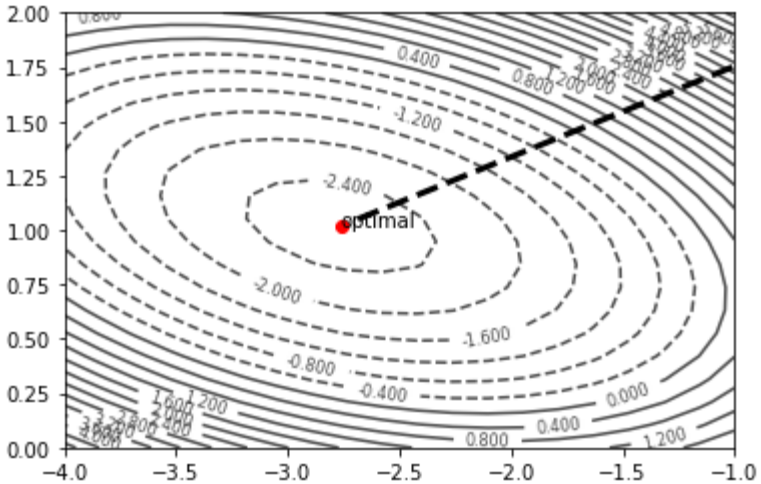
   $\mathbf{x}_{k+1} = \mathbf{x}_k - \nabla^2 f(\mathbf{x}_k)^{-1} \nabla_x f(\mathbf{x}_k)$

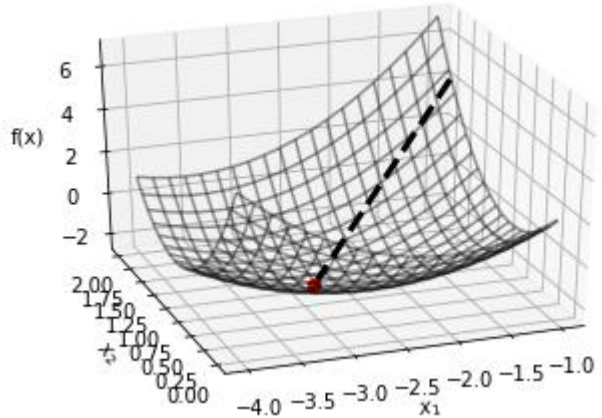
**end**

# Continuous Unconstrained Optimization – Newton’s Method

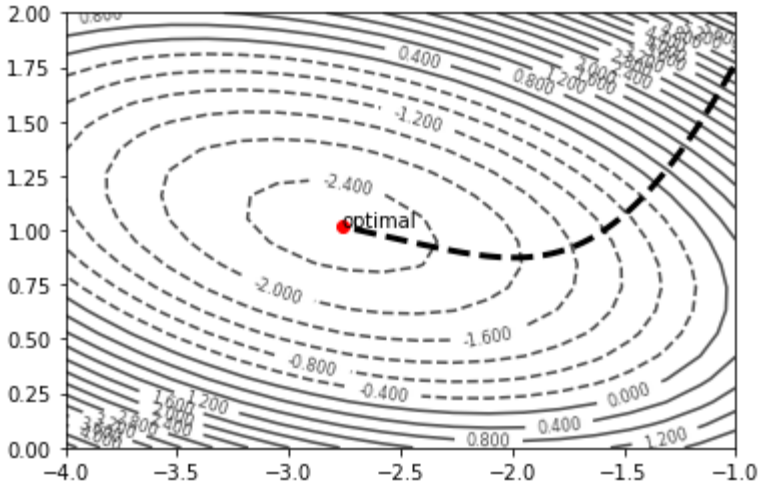
## Newton’s Method performance

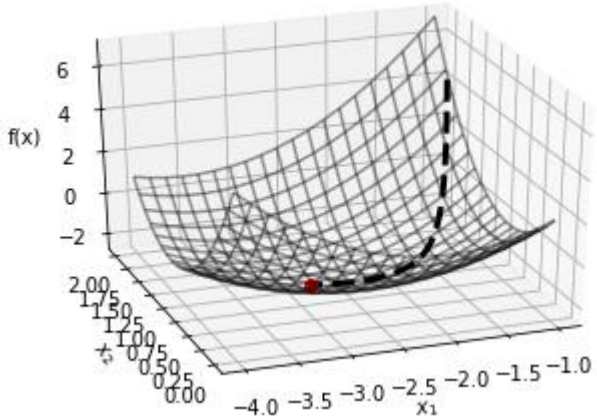
### Newton’s Method




### Gradient Descent

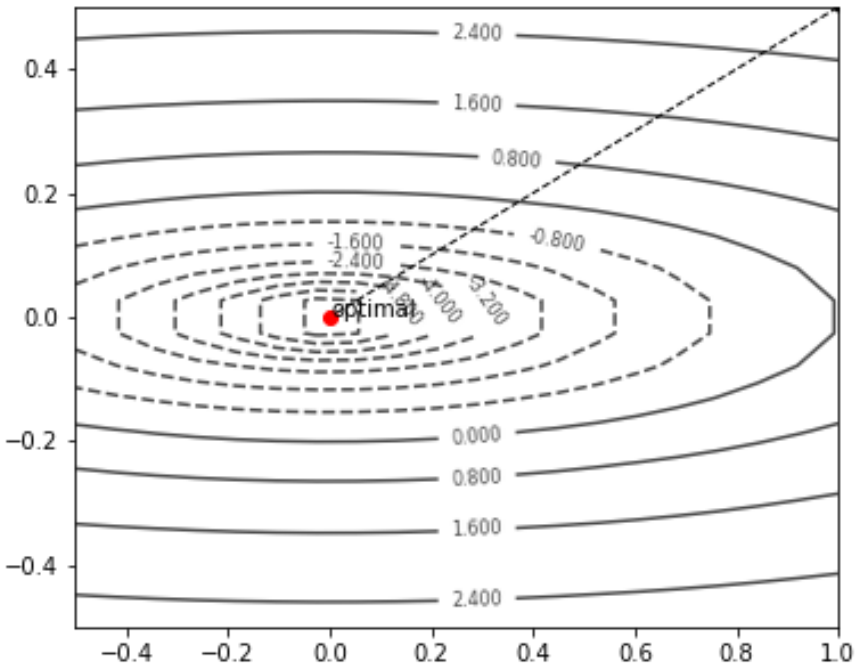


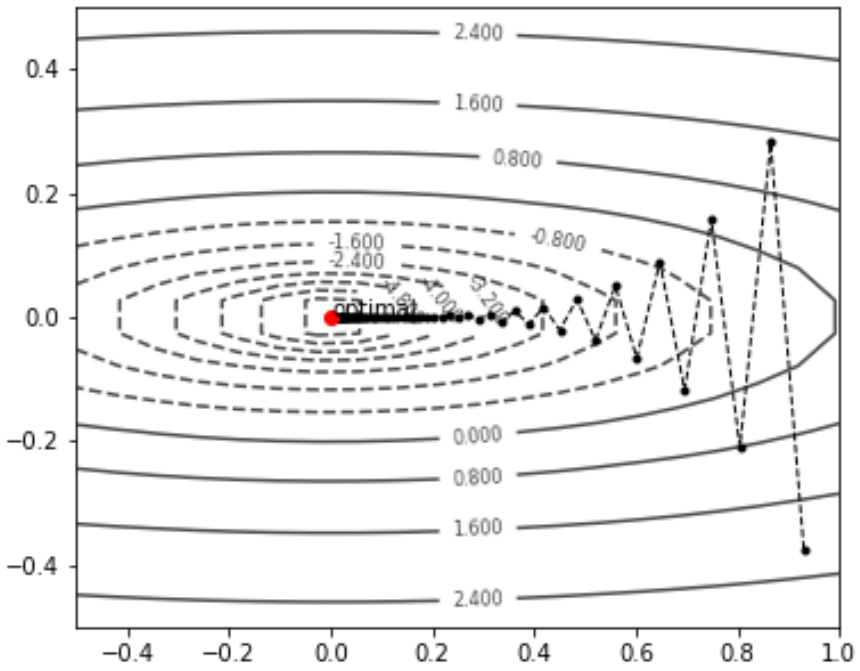
# Continuous Unconstrained Optimization – Newton’s Method

## Newton’s Method performance

### Newton’s Method




### Gradient Descent



# Continuous Unconstrained Optimization – Newton's Method

**Newton's Method key takeaways:**

- Not affected by scaling or ill conditioning.
- Second order method (uses matrices)
- The error decreases quadratically, for example:
  - $\varepsilon(k) = \frac{1}{2^{(2^k)}}, \qquad \varepsilon_{k+1} = (\varepsilon_k)^2$
  - *Once close enough to optimum
- Great for high precision
- **Used in <u>practice</u>**. Except in ANNs training.

**Disadvantages:**

- Hessian might not be invertible in nonconvex problems.*
- Expensive in computations ($n^3$) and in space ($n^2$).
- **Too expensive for Deep Learning**.

*Although you would not directly invert the Hessian for numerical reasons.

# Calculating derivatives

# Continuous Unconstrained Optimization – To do!

## Gradient Descent

Going in the direction of the most rapid function decrease

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \ \boldsymbol{\nabla}_{\boldsymbol{x}} f(\mathbf{x}_k)$$

$$\boldsymbol{\nabla}_{\boldsymbol{x}} f(\mathbf{x}_k) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}_k) \\ \vdots \\ \frac{\partial f}{\partial x_{n_x}}(\mathbf{x}_k) \end{bmatrix}$$

**Example:**

$$f(\mathbf{x}) = (x_1 - 2)^2 + (x_2 - 4)^2 + x_3 \log x_3$$

$$\boldsymbol{\nabla}_{\boldsymbol{x}} f(\mathbf{x}_k) = \begin{bmatrix} 2(x_1 - 2) \\ 2(x_2 - 4) \\ \log x_3 + 1 \end{bmatrix}$$

## How do I calculate derivatives?

In general 3 ways:

- Numerical differentiation
- Symbolic Differentiation
- Automatic Differentiation

# Continuous Unconstrained Optimization – To do!

## Gradient Descent

Going in the direction of the most rapid function decrease

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \ \boldsymbol{\nabla}_{\boldsymbol{x}} f(\mathbf{x}_k)$$

$$\boldsymbol{\nabla}_{\boldsymbol{x}} f(\mathbf{x}_k) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}_k) \\ \vdots \\ \frac{\partial f}{\partial x_{n_x}}(\mathbf{x}_k) \end{bmatrix}$$

**Example:**

$$f(\mathbf{x}) = (x_1 - 2)^2 + (x_2 - 4)^2 + x_3 \log x_3$$

$$\boldsymbol{\nabla}_{\boldsymbol{x}} f(\mathbf{x}_k) = \begin{bmatrix} 2(x_1 - 2) \\ 2(x_2 - 4) \\ \log x_3 + 1 \end{bmatrix}$$

## How do I calculate derivatives?

In general 3 ways:

- Numerical differentiation

$$f'(a) = \lim_{h \to 0} \frac{f(a+h) - f(a)}{h}.$$

$$f'(a) \approx \frac{f(a+h) - f(a)}{h}.$$

# Optimisation for Deep Learning

# Momentum

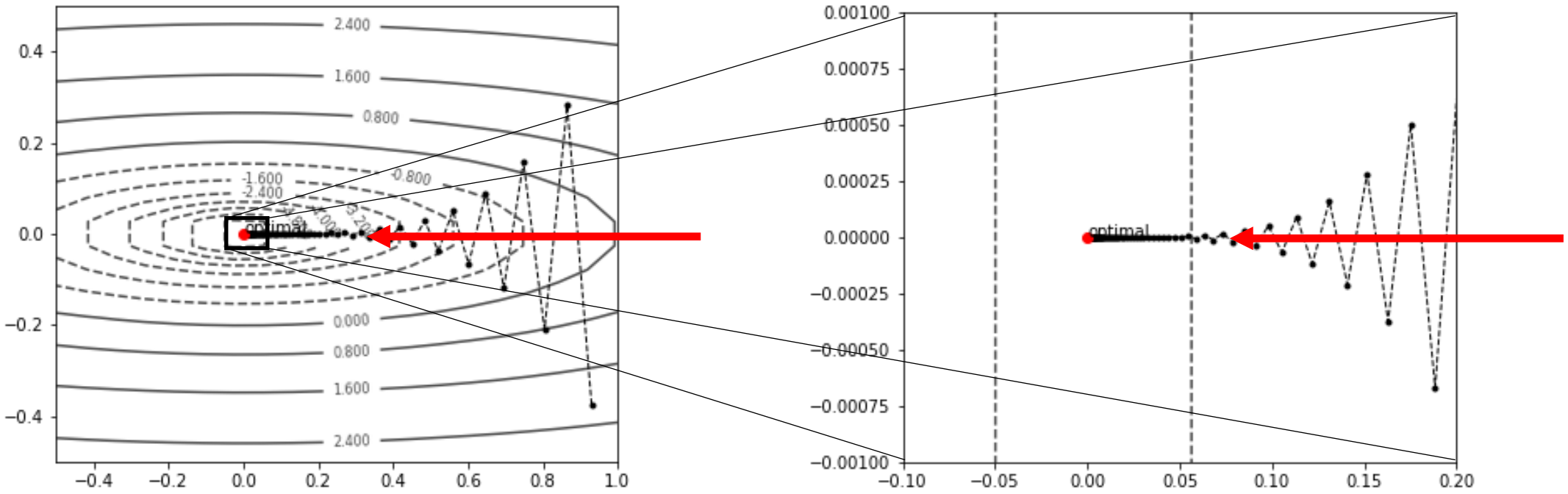
# Continuous Unconstrained Optimization – Momentum

## Momentum

## Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x}) := (x_1)^2 + (5x_2)^2$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \ \nabla f(\mathbf{x}_k)$$




Polyak, Boris T. “Some methods of speeding up the convergence of iteration methods.” *USSR Computational Mathematics and Mathematical Physics* 4, no. 5 (1964): 1-17.

# Continuous Unconstrained Optimization – Momentum

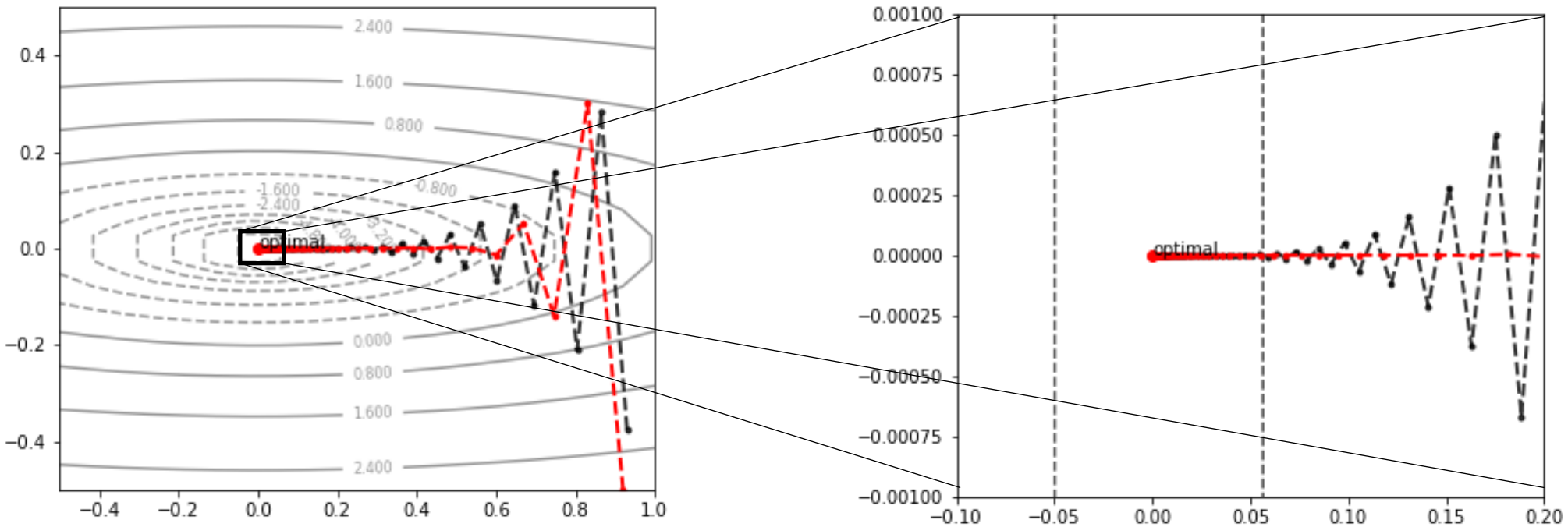
## Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x}) := (x_1)^2 + (5x_2)^2$$

## Momentum

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha\, \mathbf{g}_k(\mathbf{x}_k, \mathbf{g}_{k-1}, \beta)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha\, \nabla f(\mathbf{x}_k)$$




Polyak, Boris T. “Some methods of speeding up the convergence of iteration methods.” *USSR Computational Mathematics and Mathematical Physics* 4, no. 5 (1964): 1-17.

# Continuous Unconstrained Optimization – Momentum

## Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x}) := (x_1)^2 + (5x_2)^2$$

## Momentum

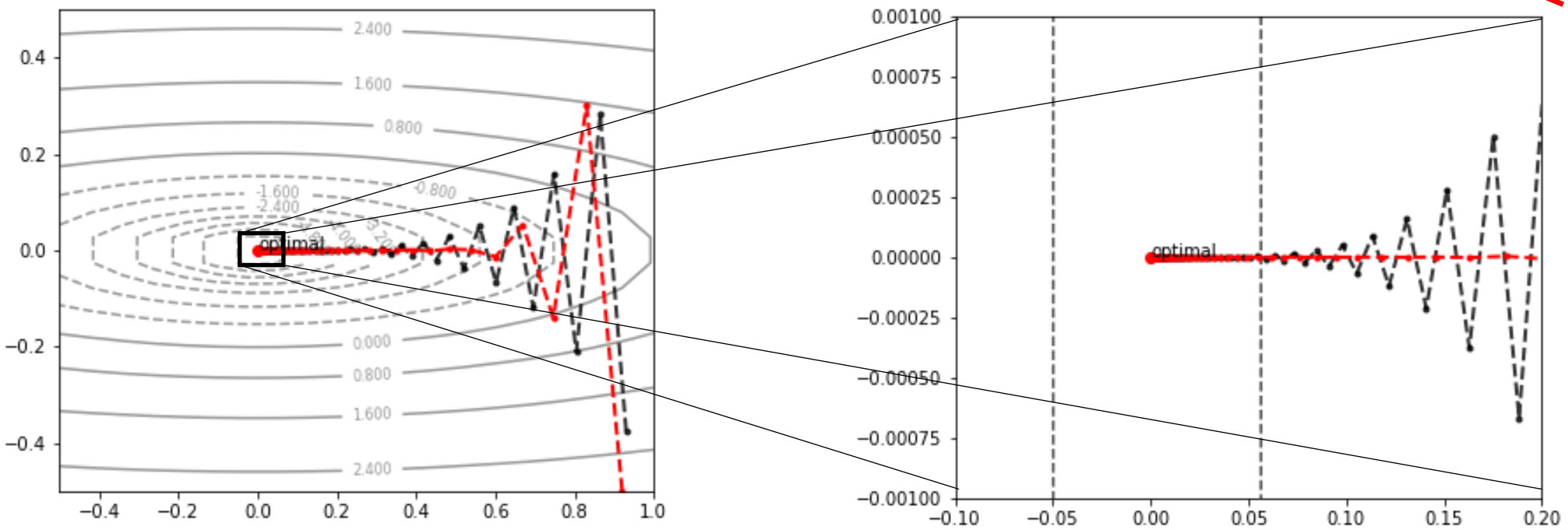
$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha\, \mathbf{g}_k(\mathbf{x}_k, \mathbf{g}_{k-1}, \beta)$$

$$\mathbf{g}_k(\mathbf{x}_k, \mathbf{g}_{k-1}, \beta) := (\beta)\boxed{\mathbf{g}_{k-1}} + (1-\beta)\boxed{\nabla_{\boldsymbol{x}} f(\mathbf{x}_k)}$$

$$\beta \in (0,1)$$

Gradient

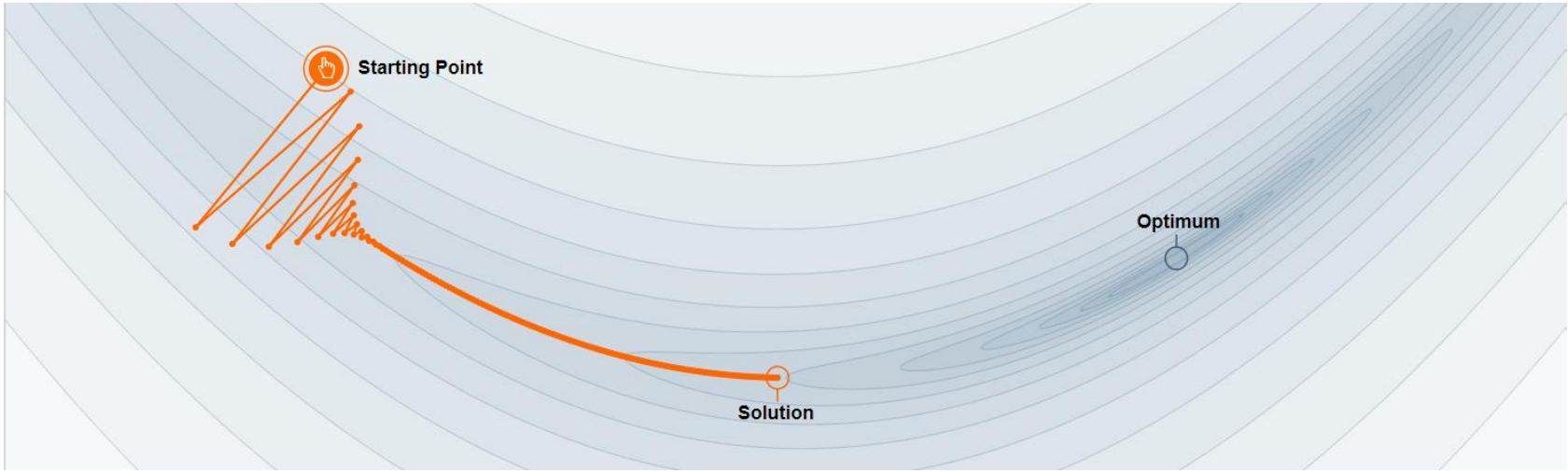
Past direction



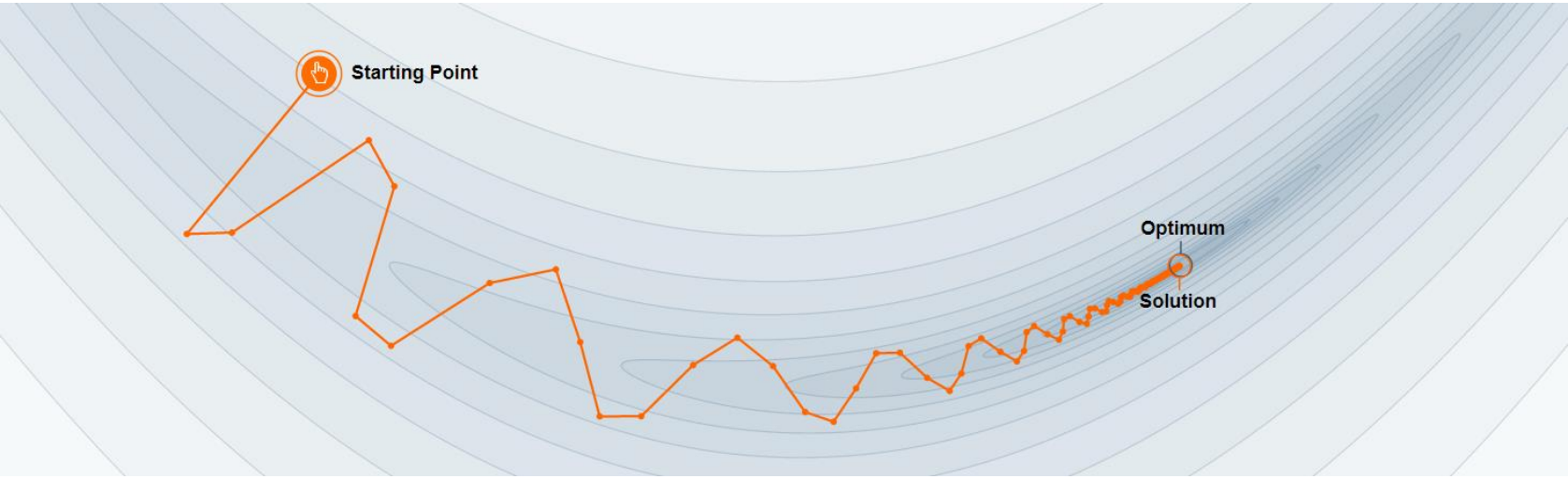

Polyak, Boris T. “Some methods of speeding up the convergence of iteration methods.” *USSR Computational Mathematics and Mathematical Physics* 4, no. 5 (1964): 1-17.

# Continuous Unconstrained Optimization – Momentum

## Gradient Descent




## Momentum



# Continuous Unconstrained Optimization – Momentum

## Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x}) \coloneqq (x_1)^2 + (5x_2)^2$$

## Momentum

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha\, \mathbf{g}_k(\mathbf{x}_k, \mathbf{g}_{k-1}, \beta)$$

$$\mathbf{g}_k(\mathbf{x}_k, \mathbf{g}_{k-1}, \beta) \coloneqq (\beta)\mathbf{g}_{k-1} + (1-\beta)\nabla_{\boldsymbol{x}} f(\mathbf{x}_k)$$

$$\beta \in (0,1)$$

**Momentum :**

- Momentum accelerates our search in the direction of minima

# Root Mean Square Propogation (RMSProp)

# Continuous Unconstrained Optimization – RSMprop

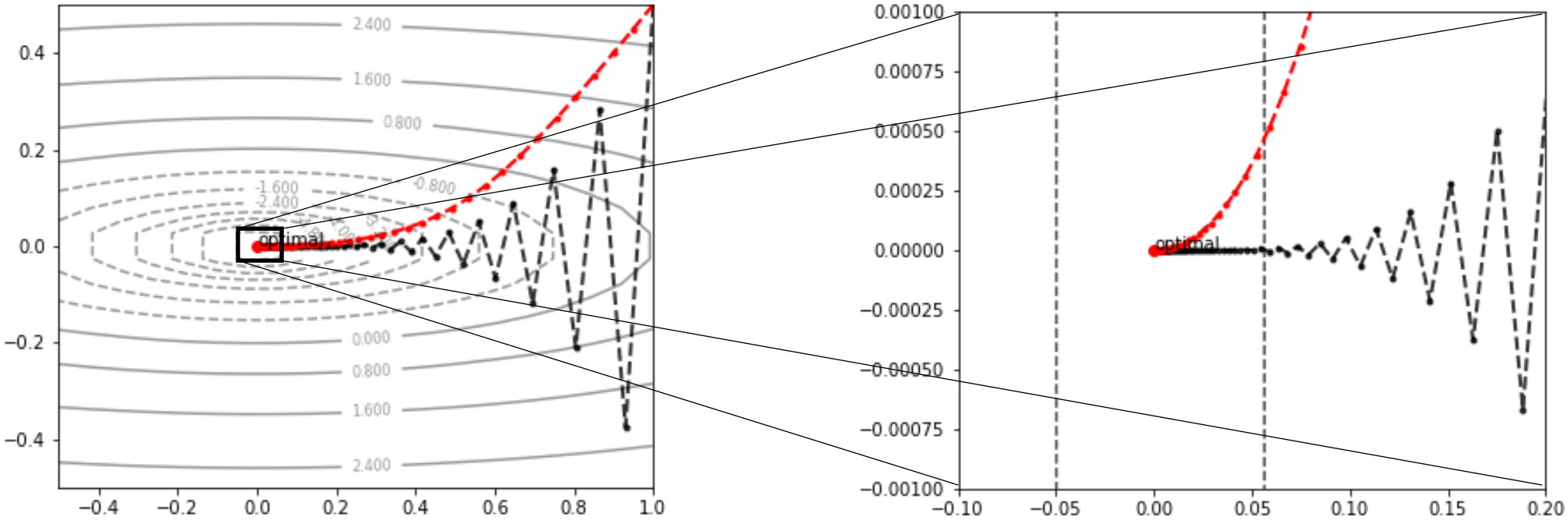
## Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x}) \coloneqq (x_1)^2 + (5x_2)^2$$

## RSMprop

Scale the gradients to avoid oscillations, which is what RSMprop does.

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \ \nabla f(\mathbf{x}_k)$$



# Continuous Unconstrained Optimization – RSMprop

## Problem statement

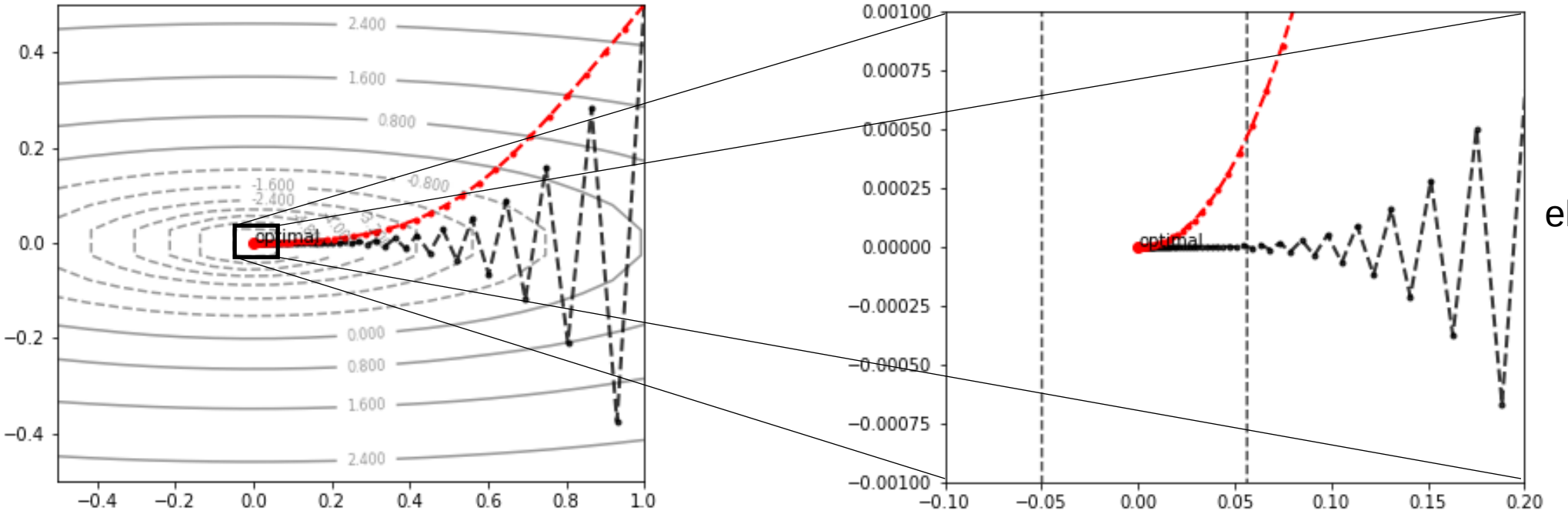
$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x}) := (x_1)^2 + (5x_2)^2$$

## RSMprop

elementwise

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \boxed{\frac{\nabla_{\boldsymbol{x}} f(\mathbf{x}_k)}{\sqrt{\mathbf{s}_k(\mathbf{x}_k, \boldsymbol{s}_{k-1}, \beta)}}} \qquad \beta \in (0,1)$$

$$\mathbf{s}_k(\mathbf{x}_k, \boldsymbol{s}_{k-1}, \beta) := (\beta)\boldsymbol{s}_{k-1} + (1-\beta)\boxed{\nabla_{\boldsymbol{x}} f(\mathbf{x}_k)^2}$$




elementwise

MM + RSMProp = Adam

Adaptive Movement Estimation

# Continuous Unconstrained Optimization – Adam

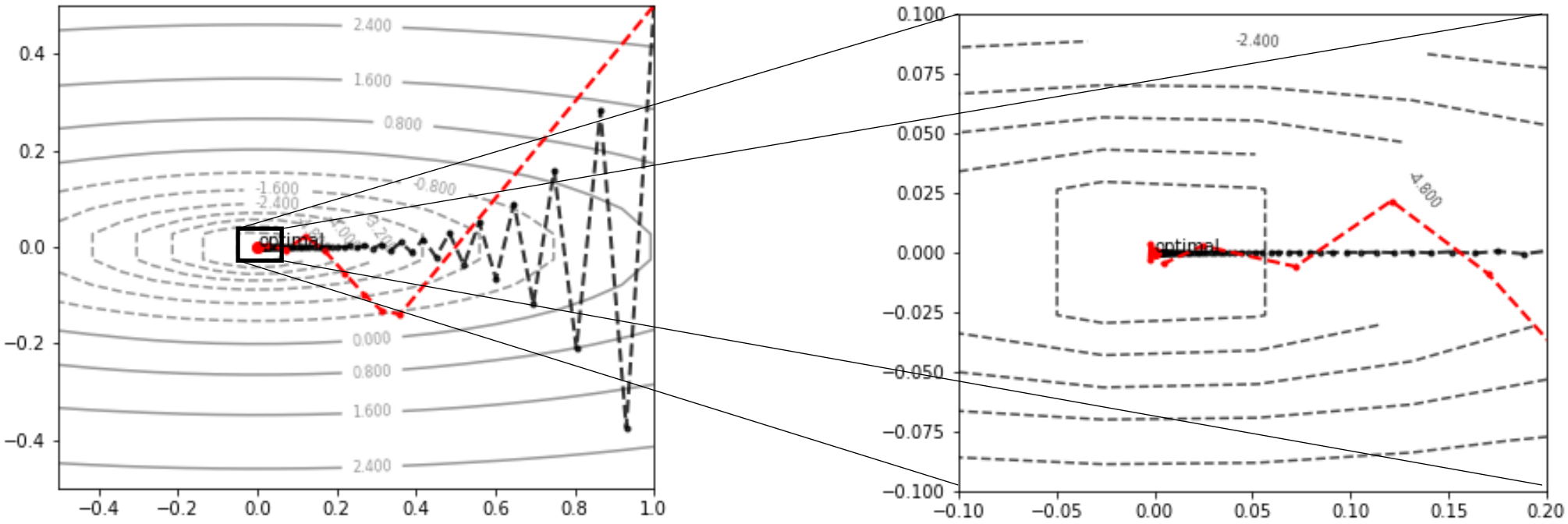
## Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x}) := (x_1)^2 + (5x_2)^2$$

## Adam

Adam combines momentum with Adaptive learning rates (RSMProp).

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \ \nabla f(\mathbf{x}_k)$$



# Continuous Unconstrained Optimization – Adam

## Problem statement

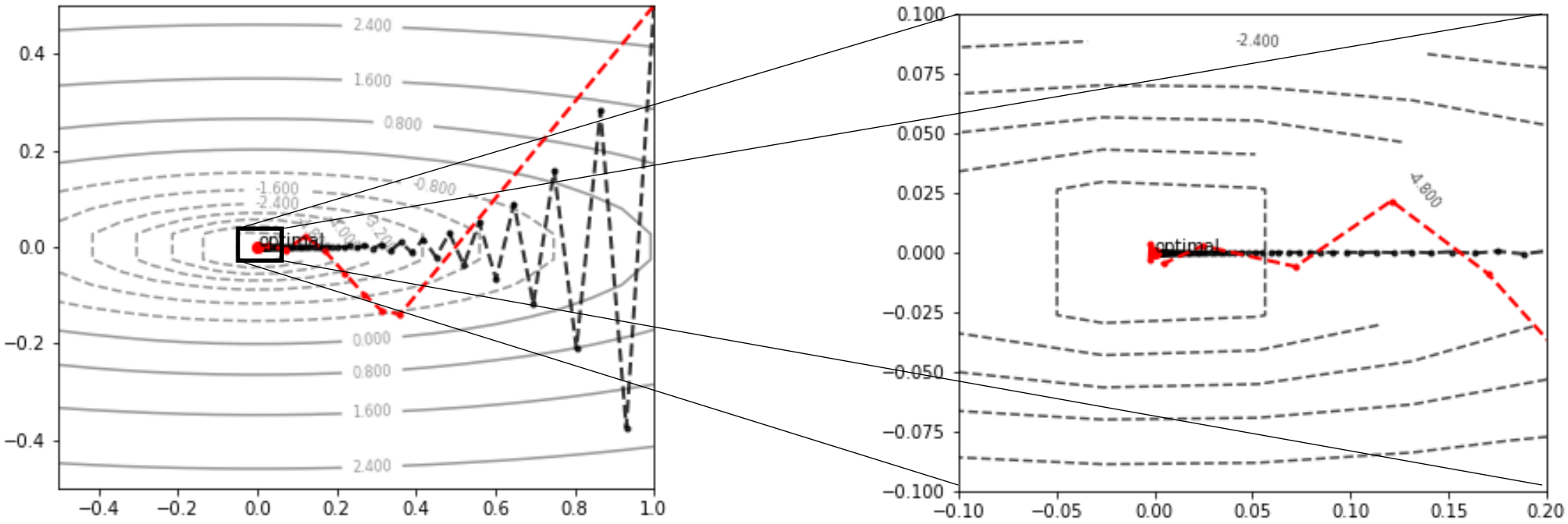
$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x}) := (x_1)^2 + (5x_2)^2$$

**Adam** → $$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \frac{\mathbf{g}_k(\mathbf{x}_k, \mathbf{g}_{k-1}, \beta_1)^{\text{corr}}}{\sqrt{\mathbf{s}_k(\mathbf{x}_k, \boldsymbol{s}_{k-1}, \beta)^{\text{corr}}}}$$

momentum → $$\mathbf{g}_k(\mathbf{x}_k, \mathbf{g}_{k-1}, \beta_1) := (\beta_1)\mathbf{g}_{k-1} + (1-\beta_1)\nabla_{\boldsymbol{x}} f(\mathbf{x}_k)$$

adaptive rate → $$\mathbf{s}_k(\mathbf{x}_k, \boldsymbol{s}_{k-1}, \beta_2) := (\beta_2)\boldsymbol{s}_{k-1} + (1-\beta_2)\nabla_{\boldsymbol{x}} f(\mathbf{x}_k)^2$$

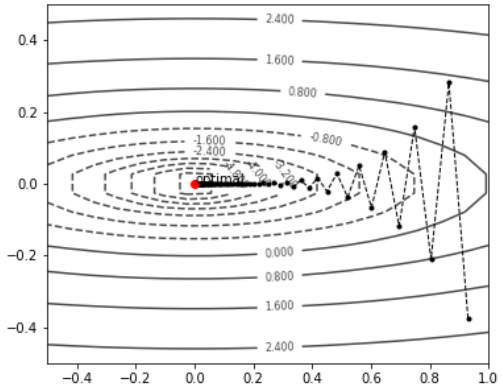
correction term
$$\begin{cases} \mathbf{g}_k(\mathbf{x}_k, \mathbf{g}_{k-1}, \beta_1)^{\text{corr}} = \mathbf{g}_k(\mathbf{x}_k, \mathbf{g}_{k-1}, \beta_1)/(1-\beta_1^i) \\ \mathbf{s}_k(\mathbf{x}_k, \mathbf{s}_{k-1}, \beta_2)^{\text{corr}} = \boldsymbol{s}_k(\mathbf{x}_k, \mathbf{s}_{k-1}, \beta_2)/(1-\beta_2^i) \end{cases}$$



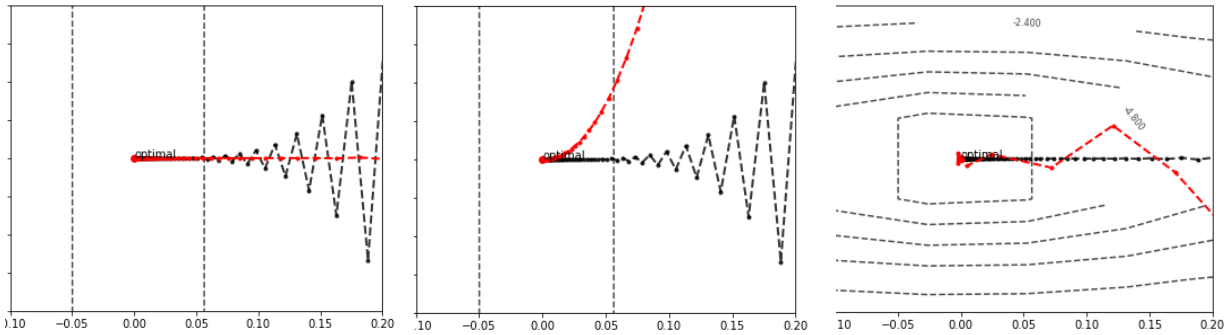
# Summary – Neural Networks

**Gradient Descent** – go-to algorithm for Deep Learning

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \ \nabla_x f(\mathbf{x}_k)$$




**Momentum** **RSMProp** **Adam**



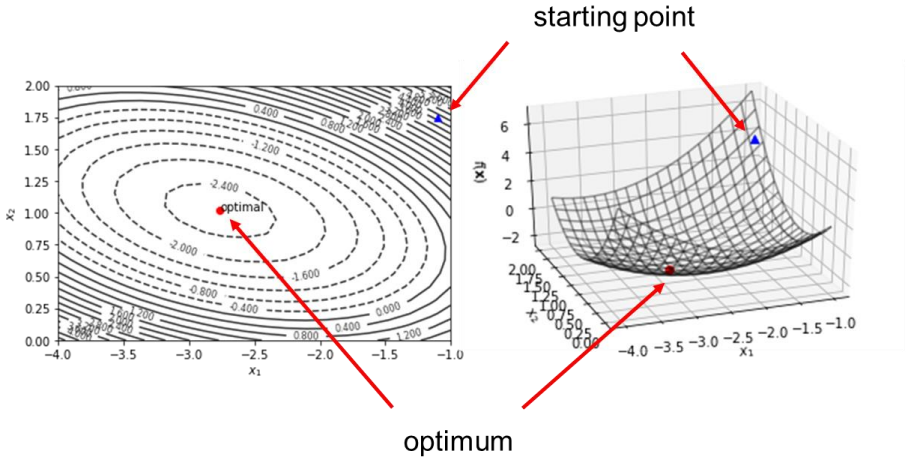

Learning is optimization

Training ANNs in Engineering application:

- NNs are generally not as large.
- First order methods are typically used
- BFGS is a good algorithm for optimization.
- It's limited memory version is still used in DL:

L-BFGS

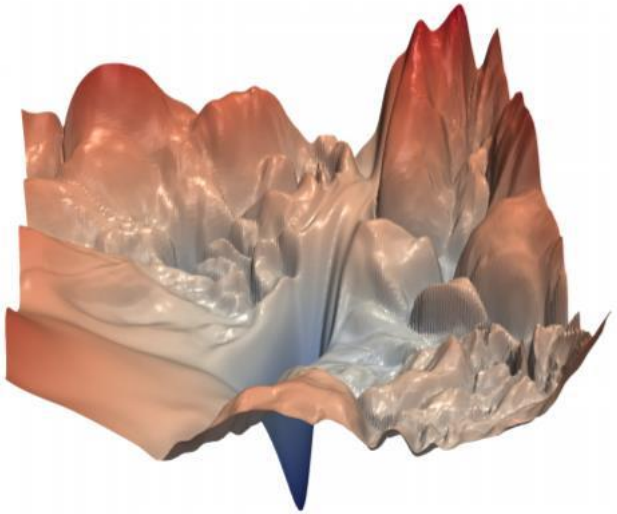
# The landscape of Neural Networks

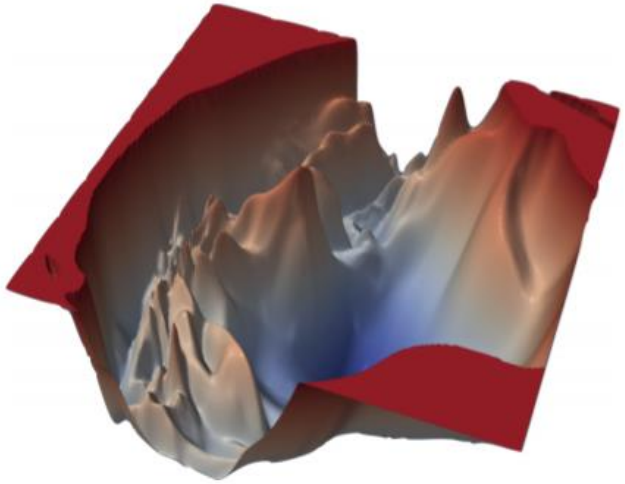


# The landscape of Neural Networks

**Visualizing the Loss Landscape of Neural Nets**

Hao Li[1], Zheng Xu[1], Gavin Taylor[2], Christoph Studer[3], Tom Goldstein[1]



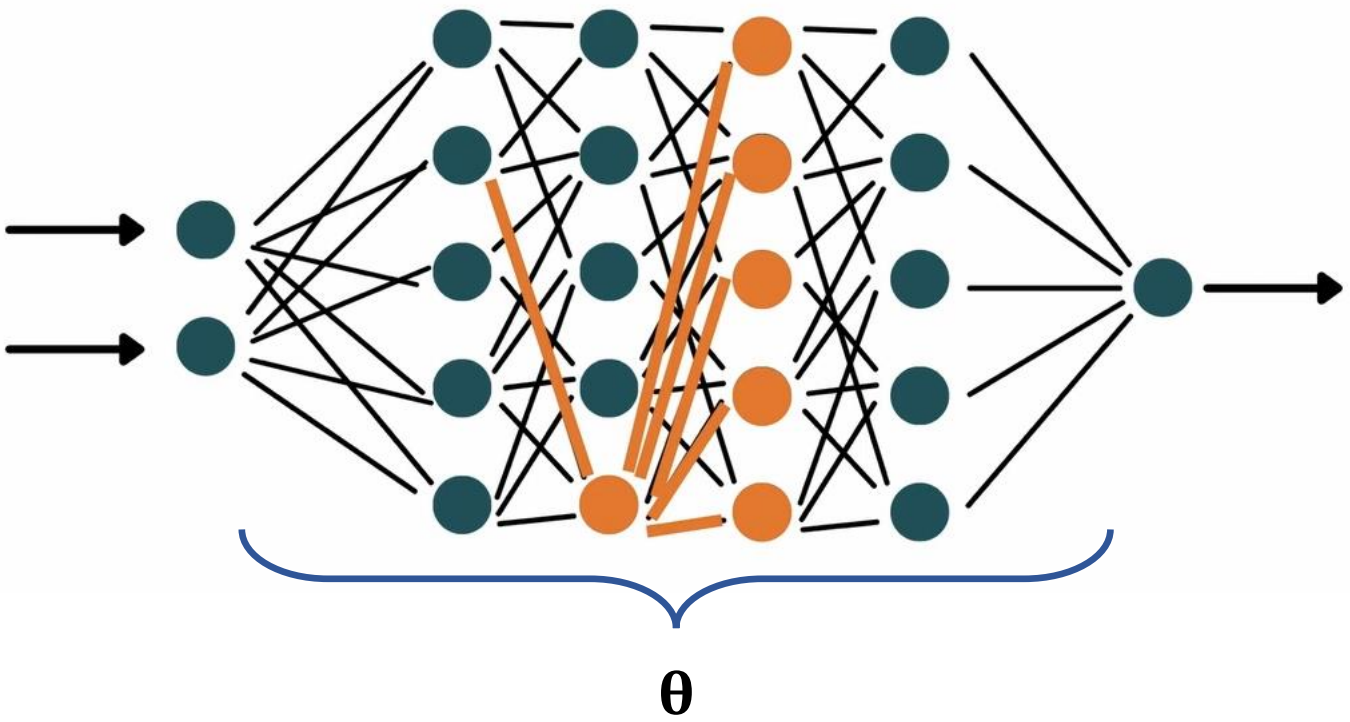


# Considerations for Neural Network training

(Vanishing and exploding gradients)

# Considerations for Neural Network training

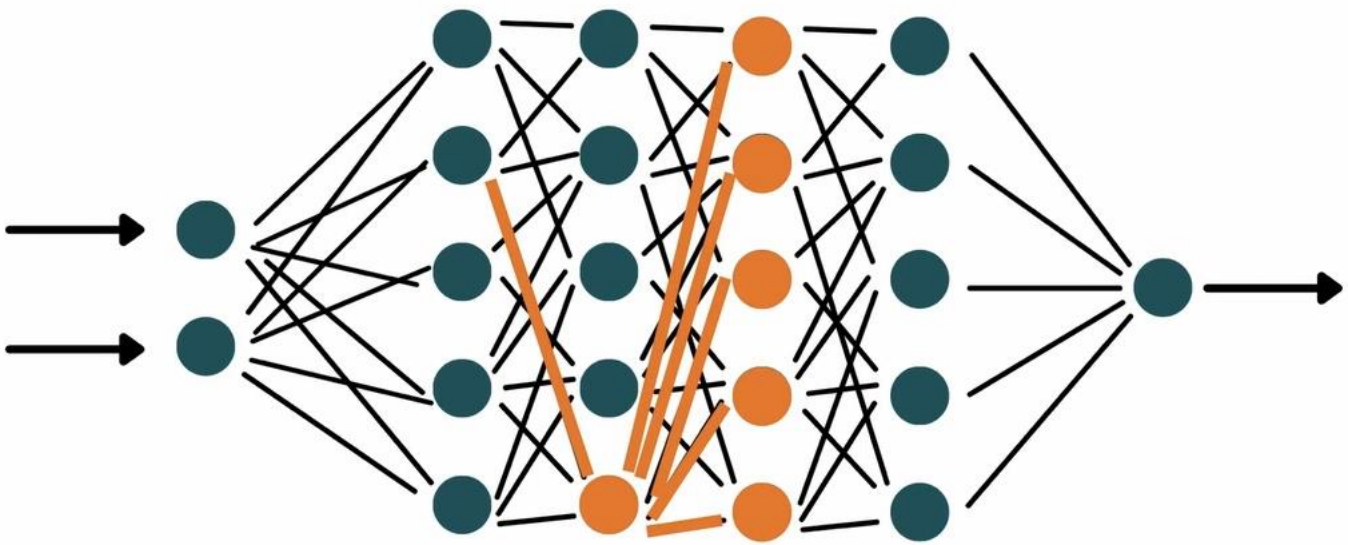
Neural network training is generally “unstable” due to vanishing or exploding gradients




$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha \ \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)$$

# Considerations for Neural Network training

Neural network training is generally “unstable” due to vanishing or exploding gradients



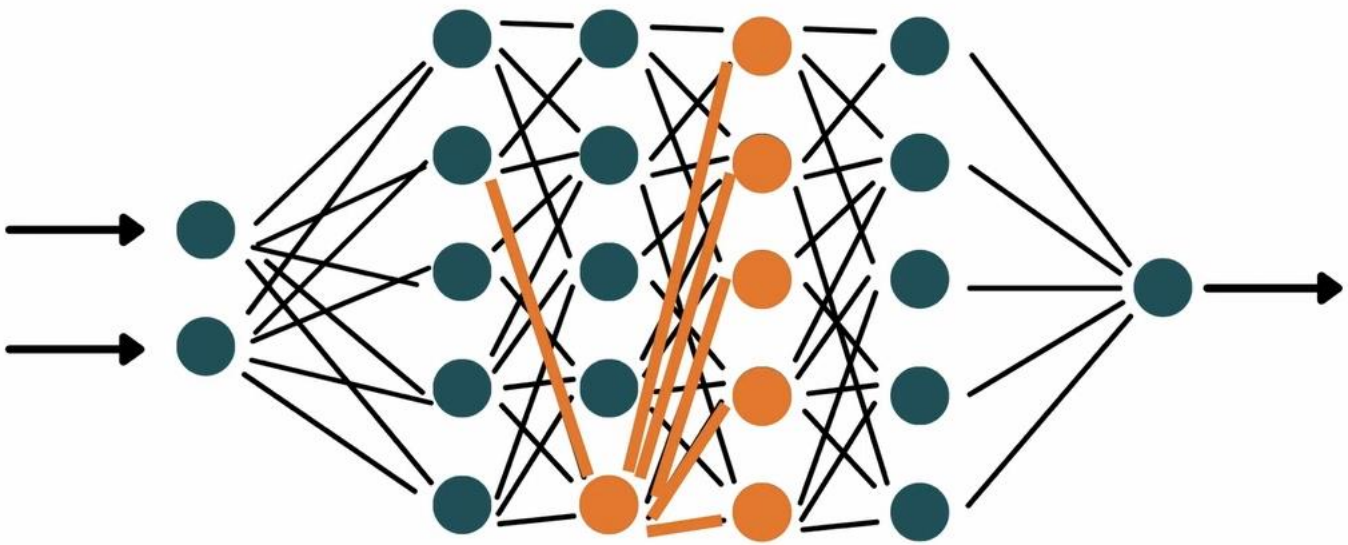
$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha \ \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)$$

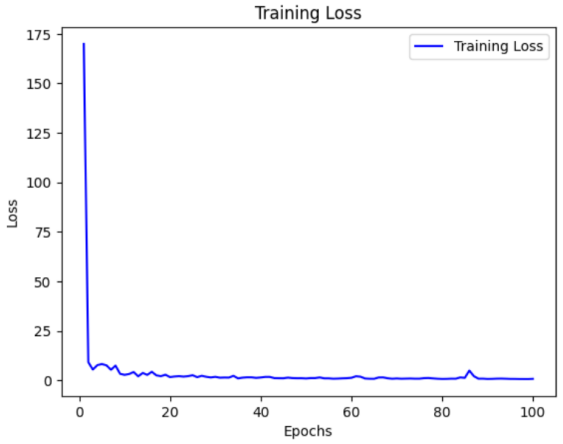
$$\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k) = \begin{bmatrix} \frac{\partial f}{\partial \theta_0} \\ \vdots \\ \frac{\partial f}{\partial \theta_{n_\theta}} \end{bmatrix}$$

These are the gradients needed to train the neural network

# Considerations for Neural Network training

Neural network training is generally “unstable” due to vanishing or exploding gradients






*vanishing gradient*

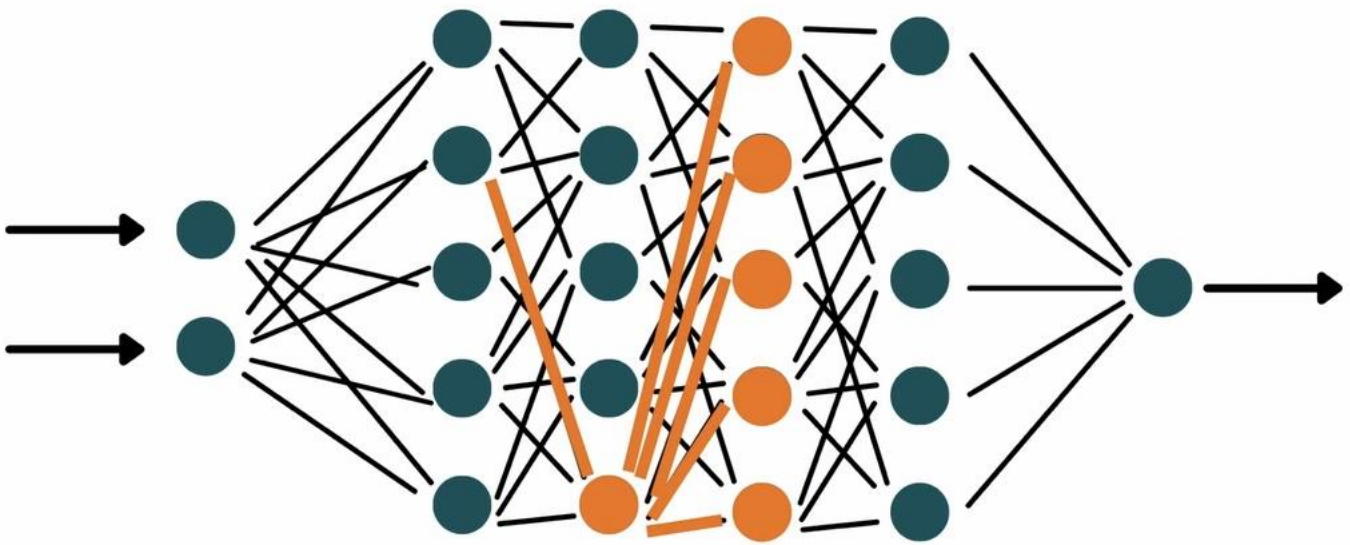
$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha \ \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)$$

$$\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k) = \begin{bmatrix} \dfrac{\partial f}{\partial \theta_0} \\ \vdots \\ \dfrac{\partial f}{\partial \theta_{n_\theta}} \end{bmatrix} \rightarrow \frac{\partial f}{\partial \theta_i} = \frac{\partial z^{(l)}}{\partial \theta_i} \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdots$$

small number X small number = very small number

# Considerations for Neural Network training

Neural network training is generally “unstable” due to vanishing or exploding gradients



*exploding gradient*

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha \ \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)$$

$$\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k) = \begin{bmatrix} \frac{\partial f}{\partial \theta_0} \\ \vdots \\ \frac{\partial f}{\partial \theta_{n_\theta}} \end{bmatrix} \rightarrow \frac{\partial f}{\partial \theta_i} = \frac{\partial z^{(l)}}{\partial \theta_i} \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdots$$

big number X big number = very big number

How do we mitigate?

# Considerations for Neural Network training

1. **How to initialize the weights (parameters of the neural network)?**

On the importance of initialization and momentum in deep learning

Ilya Sutskever[1] ILYASU@GOOGLE.COM
James Martens JMARTENS@CS.TORONTO.EDU
George Dahl GDAHL@CS.TORONTO.EDU
Geoffrey Hinton HINTON@CS.TORONTO.EDU

2. Batch Normalization

**Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift**

**Sergey Ioffe** SIOFFE@GOOGLE.COM
**Christian Szegedy** SZEGEDY@GOOGLE.COM
Google, 1600 Amphitheatre Pkwy, Mountain View, CA 94043

3. Gradient clipping

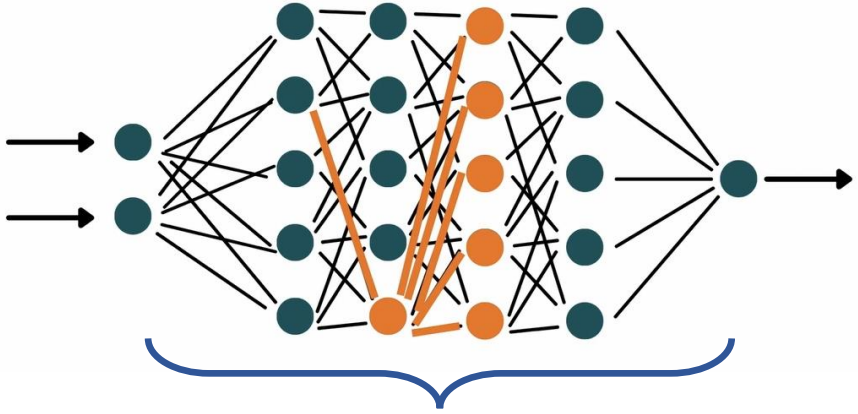
WHY GRADIENT CLIPPING ACCELERATES TRAINING: A THEORETICAL JUSTIFICATION FOR ADAPTIVITY

**Jingzhao Zhang, Tianxing He, Suvrit Sra & Ali Jadbabaie**
Massachusetts Institute of Technology
Cambridge, MA 02139, USA
{jzhzhang, tianxing, suvrit, jadbabai}@mit.edu

# Considerations for Neural Network training

## 1. How to initialize the weights (parameters of the neural network)?

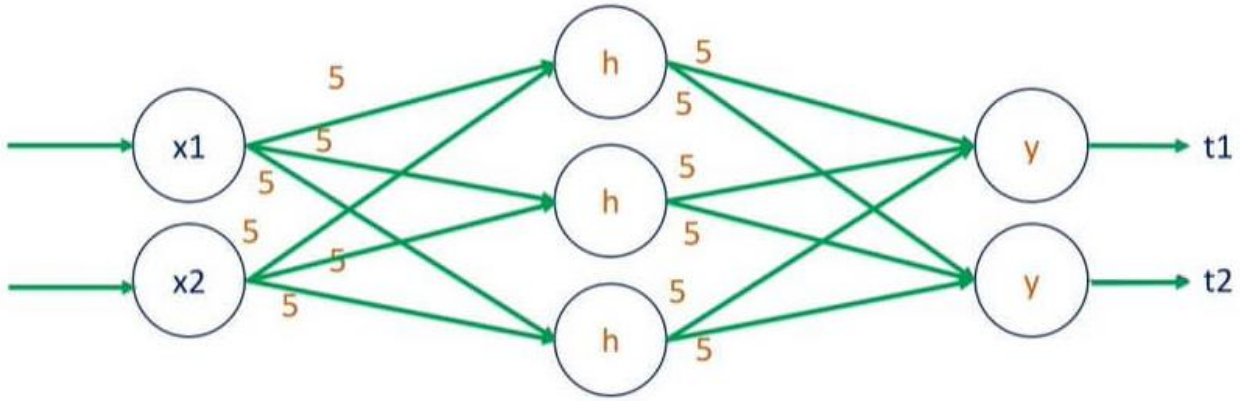
If weights are not initialized properly, the starting point $\boldsymbol{\theta}_0$ can already produce vanishing or exploding gradients $\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)$, or very close to either.

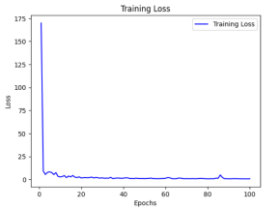


$\boldsymbol{\theta}_0$

**Symmetry problem**

Initializing all weights to the same value (whatever it is) will make all parameters act as one given the symmetry of neural networks.





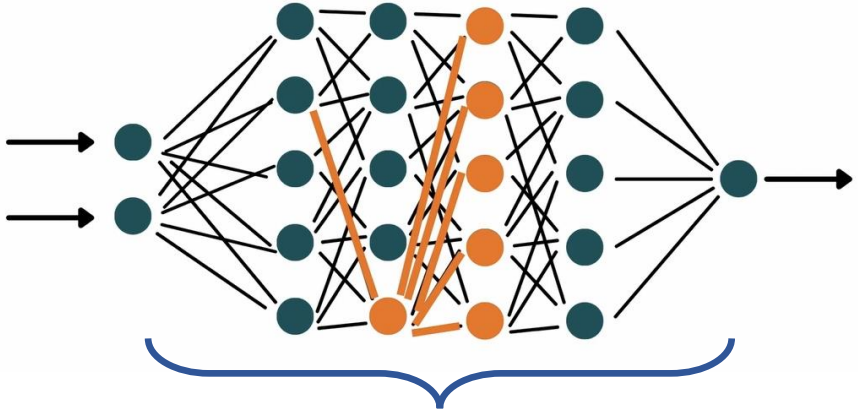
# Considerations for Neural Network training

## 1. How to initialize the weights (parameters of the neural network)?

If weights are not initialized properly, the starting point $\boldsymbol{\theta}_0$ can already produce vanishing or exploding gradients $\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)$, or very close to either.

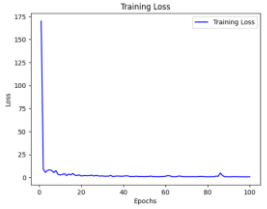
**Symmetry problem**

Initializing all weights to the same value (whatever it is) will make all parameters act as one given the symmetry of neural networks.



$\boldsymbol{\theta}_0$

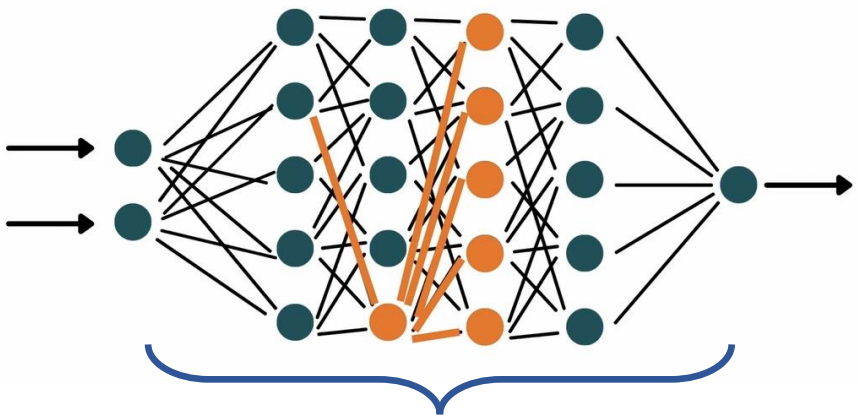
- All neurons will *update their weights in the same way*.
- All neurons will produce the same output and learn the same features.
- Limits the representational power of the network.



# Considerations for Neural Network training

## 1. How to initialize the weights (parameters of the neural network)?

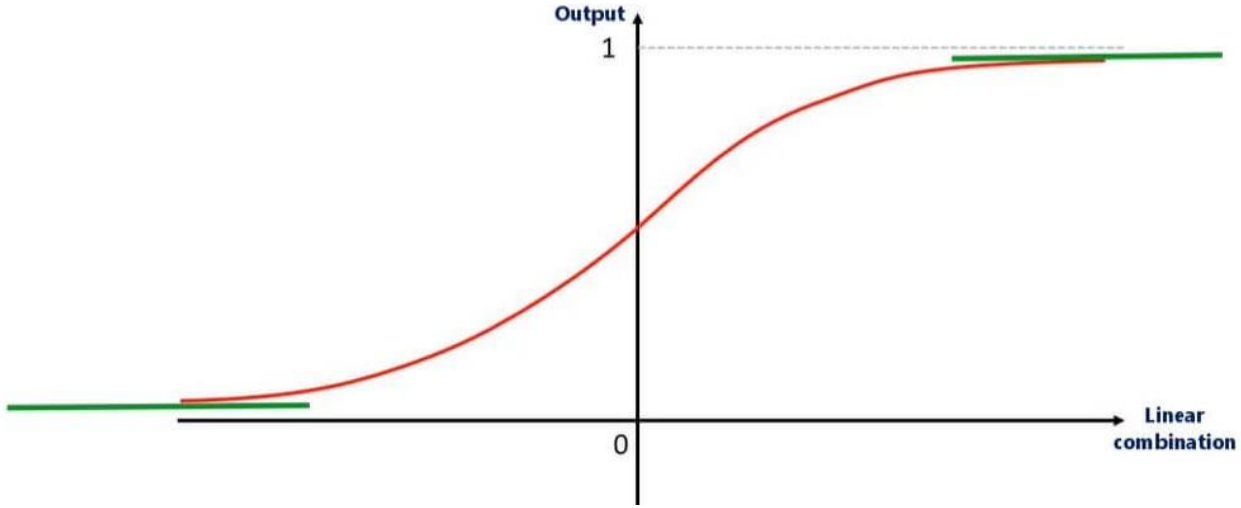
If weights are not initialized properly, the starting point $\boldsymbol{\theta}_0$ can already produce vanishing or exploding gradients $\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)$, or very close to either.

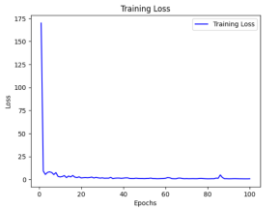


$\boldsymbol{\theta}_0$

**Initializing the values too large:**

- Will create exploding gradients.
- Output would be **only** 1s or 0s (or -1s and 1s)

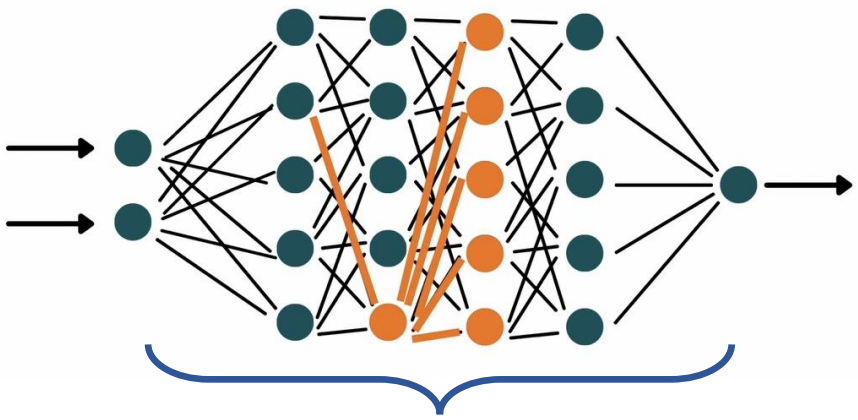




# Considerations for Neural Network training

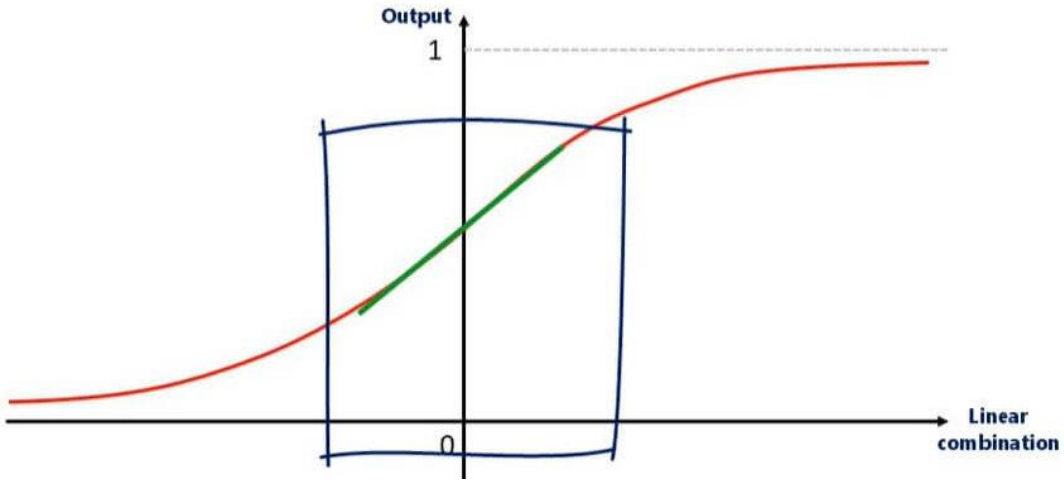
## 1. How to initialize the weights (parameters of the neural network)?

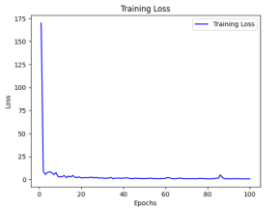
If weights are not initialized properly, the starting point $\boldsymbol{\theta}_0$ can already produce vanishing or exploding gradients $\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)$, or very close to either.




**Initializing the values too low:**

- May create vanishing gradients.
- Take advantage **only** of the linear part of the network



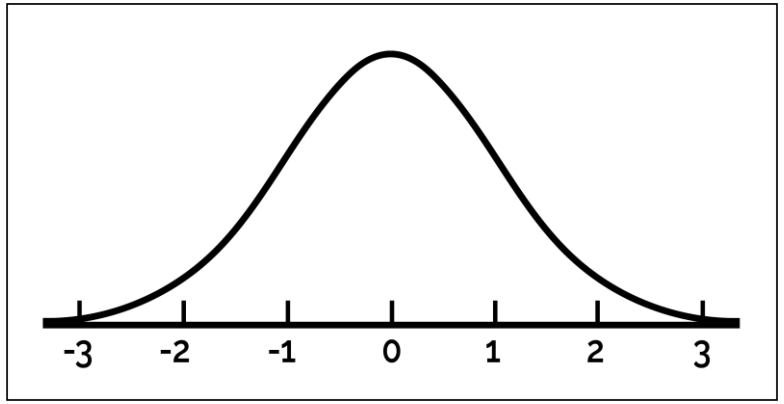


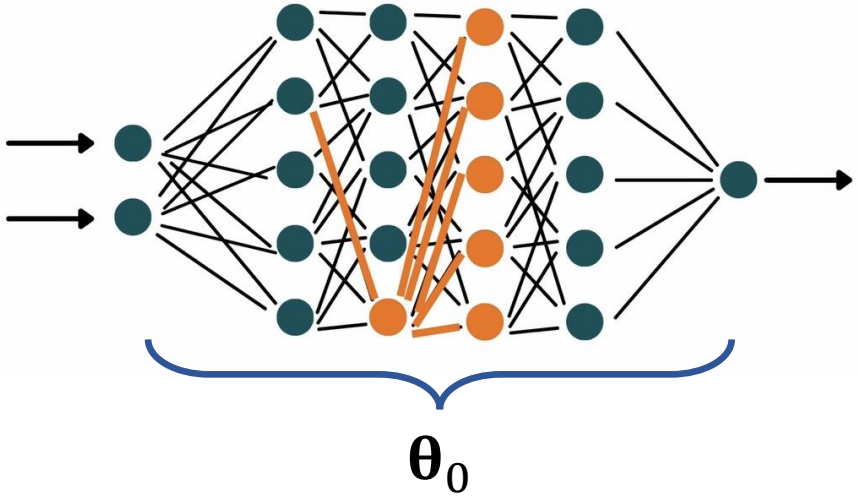
# Considerations for Neural Network training

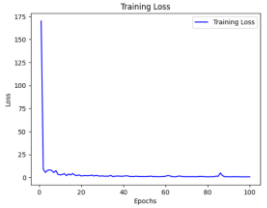
## 1. How to initialize the weights (parameters of the neural network)?

If weights are not initialized properly, the starting point $\boldsymbol{\theta}_0$ can already produce vanishing or exploding gradients $\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)$, or very close to either.

**Normal Xavier initialization**:






Initialize weights randomly from a Gaussian distribution:

$$\mu = 0, \qquad \sigma^2 = \frac{2}{\text{inputs} + \text{outputs}}$$

Note: this is good for tanh. *He initialization* good for ReLU

# Considerations for Neural Network training

1. How to initialize the weights (parameters of the neural network)?

On the importance of initialization and momentum in deep learning

Ilya Sutskever[1] ILYASU@GOOGLE.COM
James Martens JMARTENS@CS.TORONTO.EDU
George Dahl GDAHL@CS.TORONTO.EDU
Geoffrey Hinton HINTON@CS.TORONTO.EDU

**2. Batch Normalization**

**Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift**

**Sergey Ioffe** SIOFFE@GOOGLE.COM
**Christian Szegedy** SZEGEDY@GOOGLE.COM
Google, 1600 Amphitheatre Pkwy, Mountain View, CA 94043

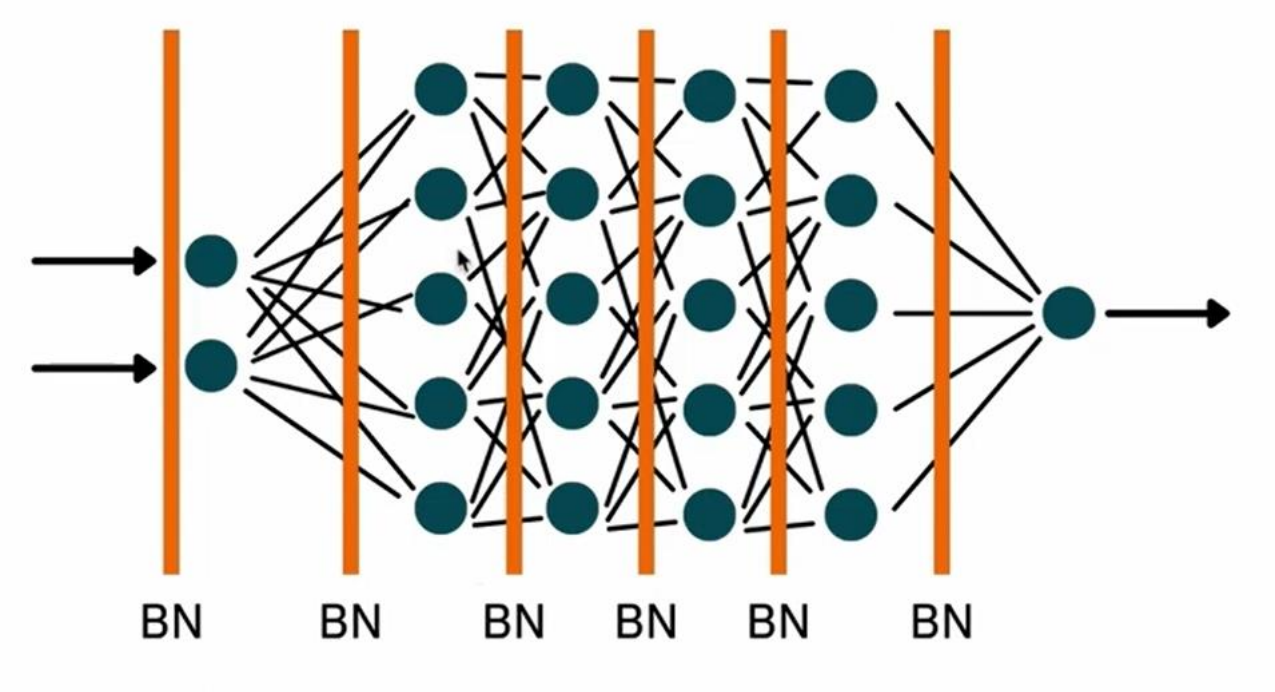
3. Gradient clipping

WHY GRADIENT CLIPPING ACCELERATES TRAINING: A THEORETICAL JUSTIFICATION FOR ADAPTIVITY

**Jingzhao Zhang, Tianxing He, Suvrit Sra & Ali Jadbabaie**
Massachusetts Institute of Technology
Cambridge, MA 02139, USA
{jzhzhang, tianxing, suvrit, jadbabai}@mit.edu

# Considerations for Neural Network training

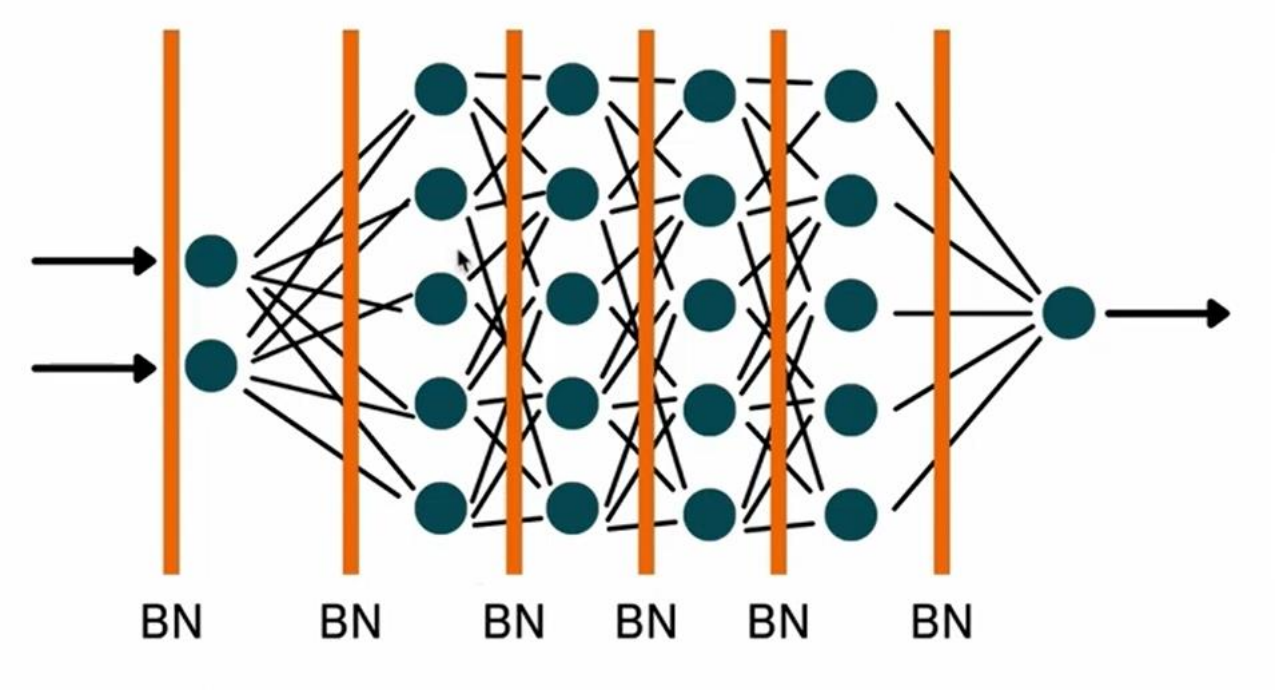
## 2. Batch Normalization




- ‘standardize at every layer’
- This is a ‘transformation’ of sorts
- Extra benefits:
  - May have better performance.
  - Reduces need for regularization.
  - Faster training convergence.

# Considerations for Neural Network training

## 2. Batch Normalization




- **Layer Normalization** can work even better than batch normalization.

**Layer Normalization**

**Jimmy Lei Ba**
University of Toronto
jimmy@psi.toronto.edu

**Jamie Ryan Kiros**
University of Toronto
rkiros@cs.toronto.edu

**Geoffrey E. Hinton**
University of Toronto
and Google Inc.
hinton@cs.toronto.edu

# Considerations for Neural Network training

1. How to initialize the weights (parameters of the neural network)?

On the importance of initialization and momentum in deep learning

Ilya Sutskever[1] ILYASU@GOOGLE.COM
James Martens JMARTENS@CS.TORONTO.EDU
George Dahl GDAHL@CS.TORONTO.EDU
Geoffrey Hinton HINTON@CS.TORONTO.EDU

2. Batch Normalization

**Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift**

**Sergey Ioffe** SIOFFE@GOOGLE.COM
**Christian Szegedy** SZEGEDY@GOOGLE.COM
Google, 1600 Amphitheatre Pkwy, Mountain View, CA 94043

3. Gradient clipping

WHY GRADIENT CLIPPING ACCELERATES TRAINING: A THEORETICAL JUSTIFICATION FOR ADAPTIVITY

**Jingzhao Zhang, Tianxing He, Suvrit Sra & Ali Jadbabaie**
Massachusetts Institute of Technology
Cambridge, MA 02139, USA
{jzhzhang, tianxing, suvrit, jadbabai}@mit.edu

# Considerations for Neural Network training

1. How to initialize the weights (parameters of the neural network)?

On the importance of initialization and momentum in deep learning

Ilya Sutskever[1] ILYASU@GOOGLE.COM
James Martens JMARTENS@CS.TORONTO.EDU
George Dahl GDAHL@CS.TORONTO.EDU
Geoffrey Hinton HINTON@CS.TORONTO.EDU

2. Batch Normalization

**Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift**

**Sergey Ioffe** SIOFFE@GOOGLE.COM
**Christian Szegedy** SZEGEDY@GOOGLE.COM
Google, 1600 Amphitheatre Pkwy, Mountain View, CA 94043

**3. Gradient clipping**

WHY GRADIENT CLIPPING ACCELERATES TRAINING: A THEORETICAL JUSTIFICATION FOR ADAPTIVITY

**Jingzhao Zhang, Tianxing He, Suvrit Sra & Ali Jadbabaie**
Massachusetts Institute of Technology
Cambridge, MA 02139, USA
{jzhzhang, tianxing, suvrit, jadbabai}@mit.edu

# Considerations for Neural Network training

## 3. Gradient clipping

Used against exploding gradients.

What happens if one or more gradient components are too large:

$$\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k) = \begin{bmatrix} \frac{\partial f}{\partial \theta_0} \\ \vdots \\ \frac{\partial f}{\partial \theta_{n_\theta}} \end{bmatrix}$$

**Value-based Clipping**

Individual gradient values are clipped to lie within a range.

if a gradient value $\frac{\partial f}{\partial \theta_i}$ is larger than $\tau$, it is set to $\tau$; if it is less than a minimum value $-\tau$ set to $-\tau$.

Con: gradient direction is distorted.

**Norm-based Clipping**

Gradients are rescaled if their norm exceeds a predefined threshold.

if the norm of the gradient $\|\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)\|$ exceeds a threshold $\tau$, it is rescaled to

$$\mathbf{g}_k \leftarrow \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k) \cdot \frac{\tau}{\|\nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}_k)\|}$$

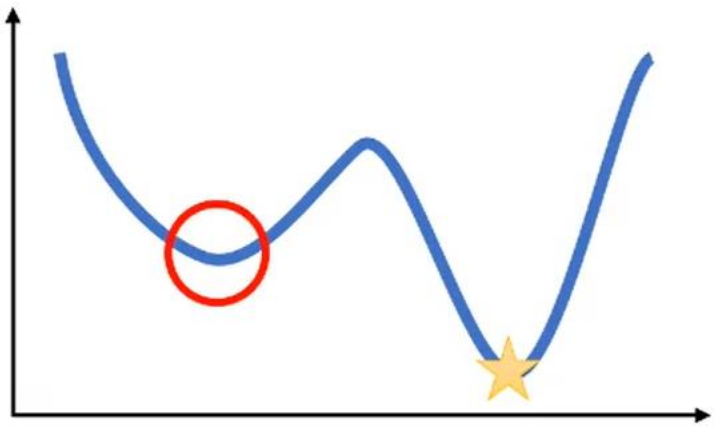
Con: some values can be very small

# Three undesirable stationary points in NN training

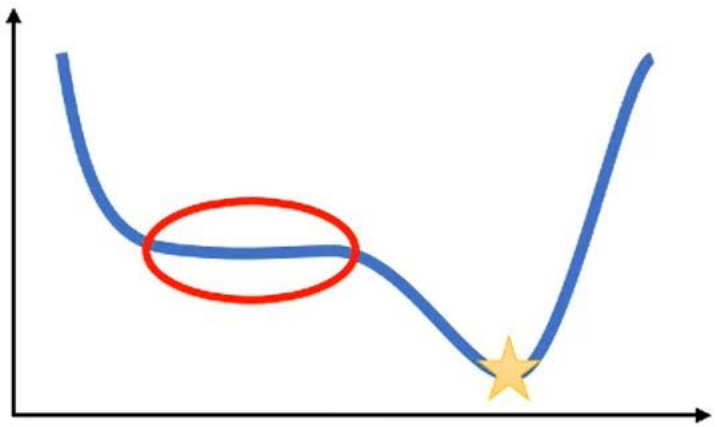
*Deep Learning course by Sergey Levine CS 182 2021, Lecture 4: Optimization

# The landscape of Neural Networks – three stationary point

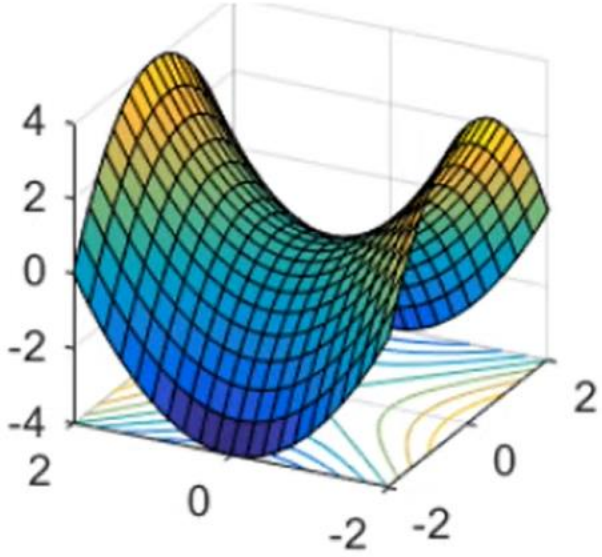
**<u>Three undesirable stationary points in NN training</u>**



Local optimum



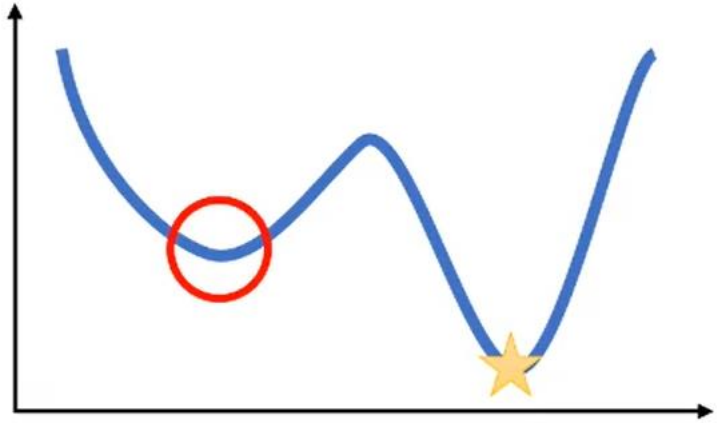
Plateau




Saddle point

*Deep Learning course by Sergey Levine CS 182 2021, Lecture 4: Optimization

# The landscape of Neural Networks – local optimum

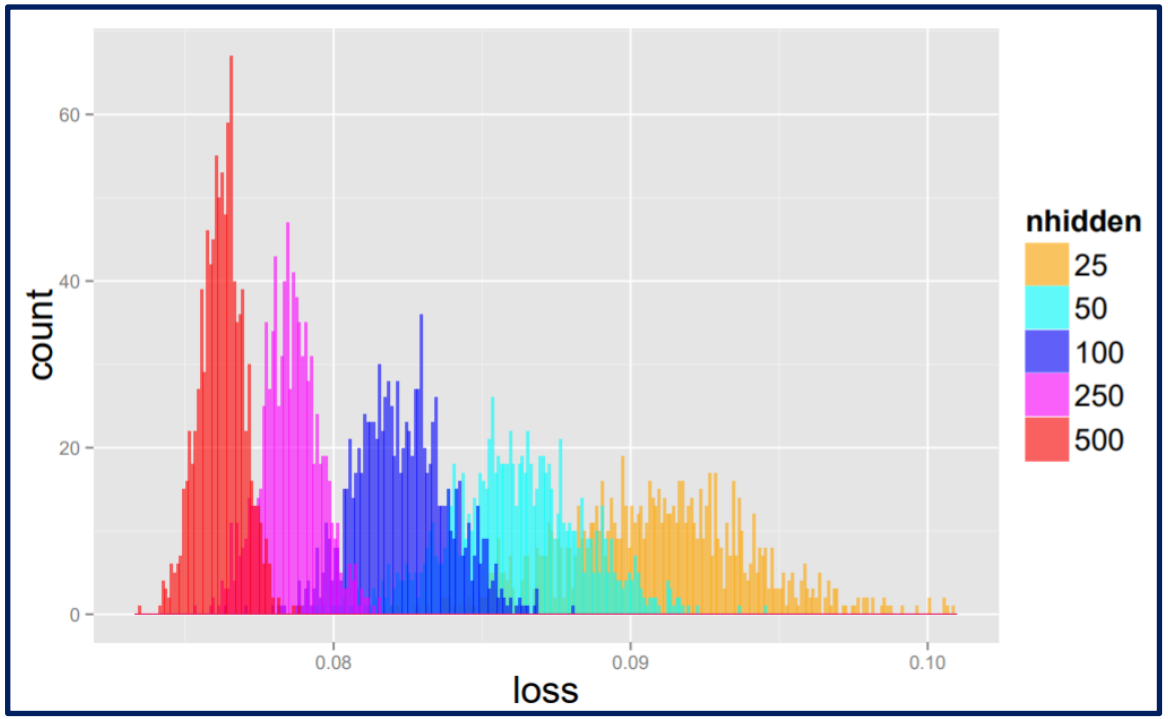


Local optimum

- Used to be the major concern for optimization of NNs
- Less of an issue as NNs become larger
- For big networks local minima are almost as Good as global

The Loss Surfaces of Multilayer Networks

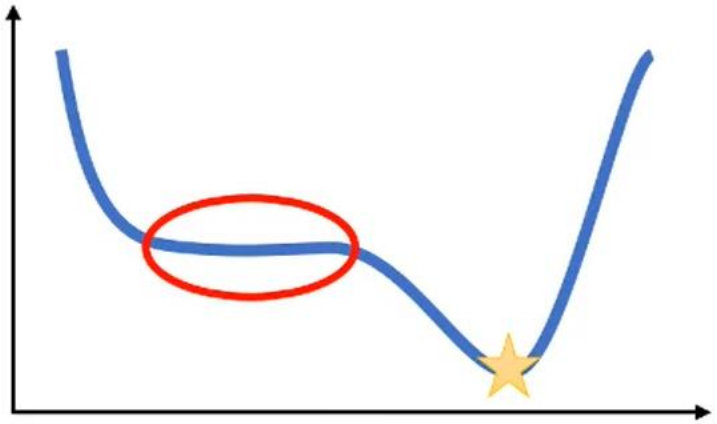
Anna Choromanska achoroma@cims.nyu.edu
Mikael Henaff mbh305@nyu.edu
Michael Mathieu mathieu@cs.nyu.edu
Gérard Ben Arous benarous@cims.nyu.edu
Yann LeCun yann@cs.nyu.edu



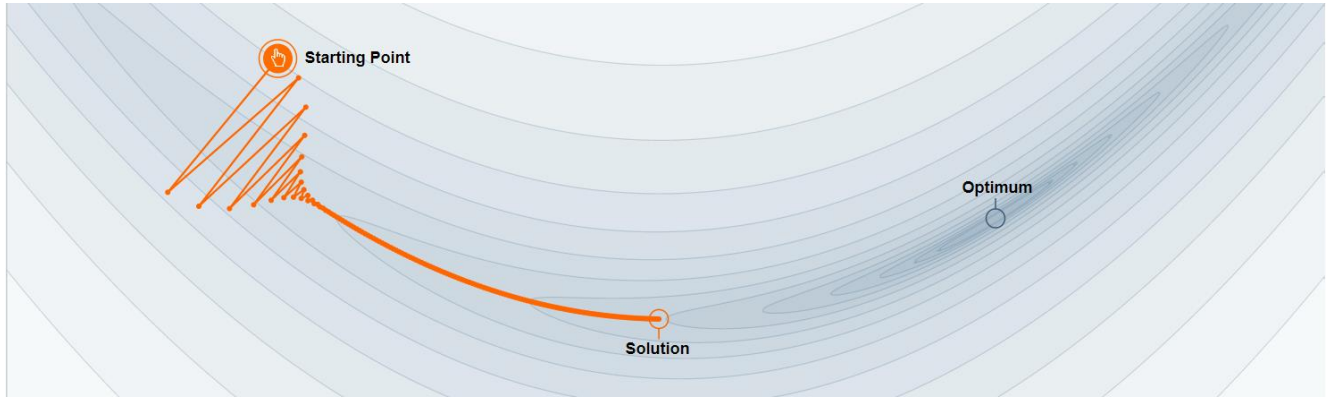

*Deep Learning course by Sergey Levine CS 182 2021, Lecture 4: Optimization

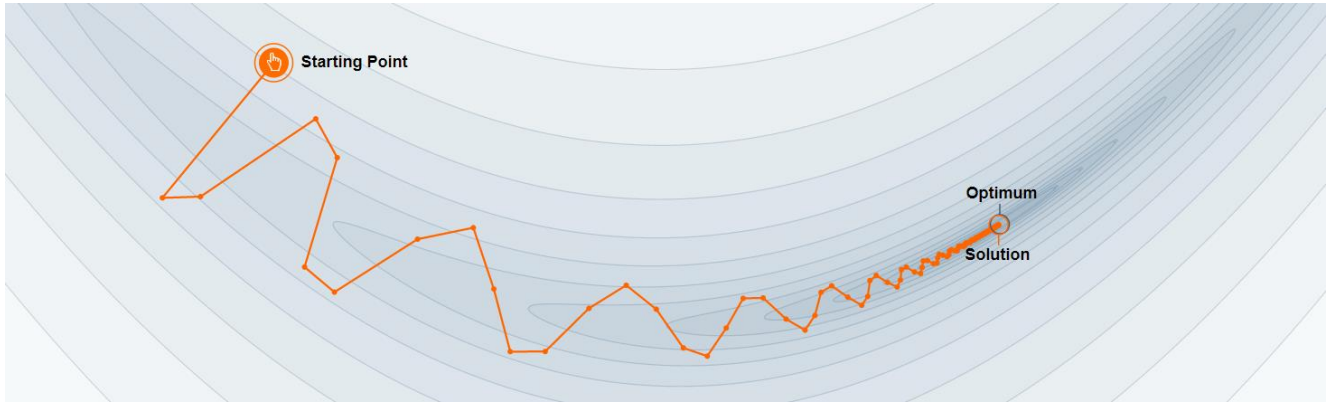
# The landscape of Neural Networks – plateau

- Are a big problem for vanilla gradient descent
- Learning rate should be large to get over it (but not explode)
- Momentum helps address this



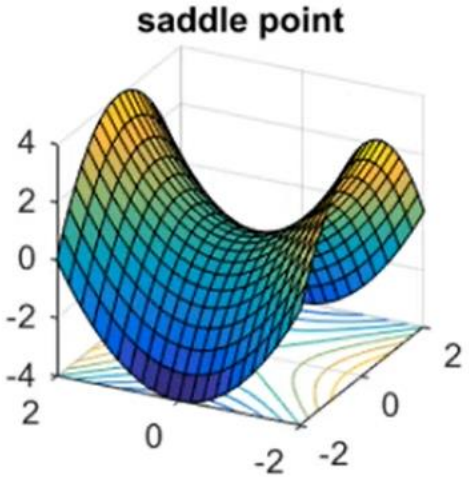
plateau

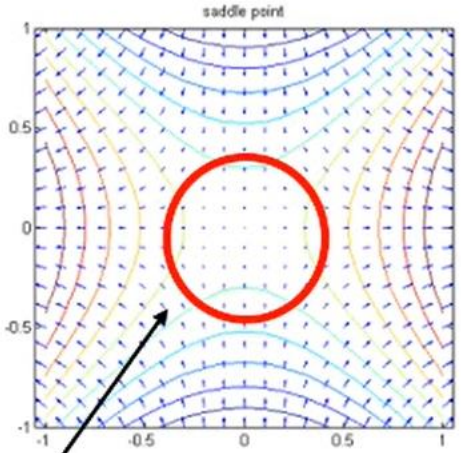


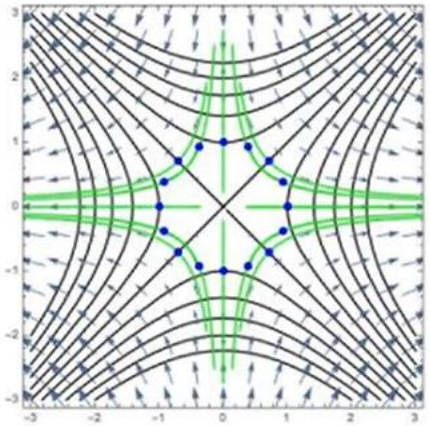

*Deep Learning course by Sergey Levine CS 182 2021, Lecture 4: Optimization

# The landscape of Neural Networks – saddle points




Gradient vectors



Gradient flows (green)

the gradient here is very small

**Most stationary points in NNs are saddle points!**

updates

**First-order methods almost always avoid strict saddle points**

Jason D. Lee[1] · Ioannis Panageas[2] · Georgios Piliouras[3] · Max Simchowitz[4] · Michael I. Jordan[4] · Benjamin Recht[4]

**Gradient Descent Can Take Exponential Time to Escape Saddle Points**

**Simon S. Du**
Carnegie Mellon University
ssdu@cs.cmu.edu

**Chi Jin**
University of California, Berkeley
chijin@berkeley.edu

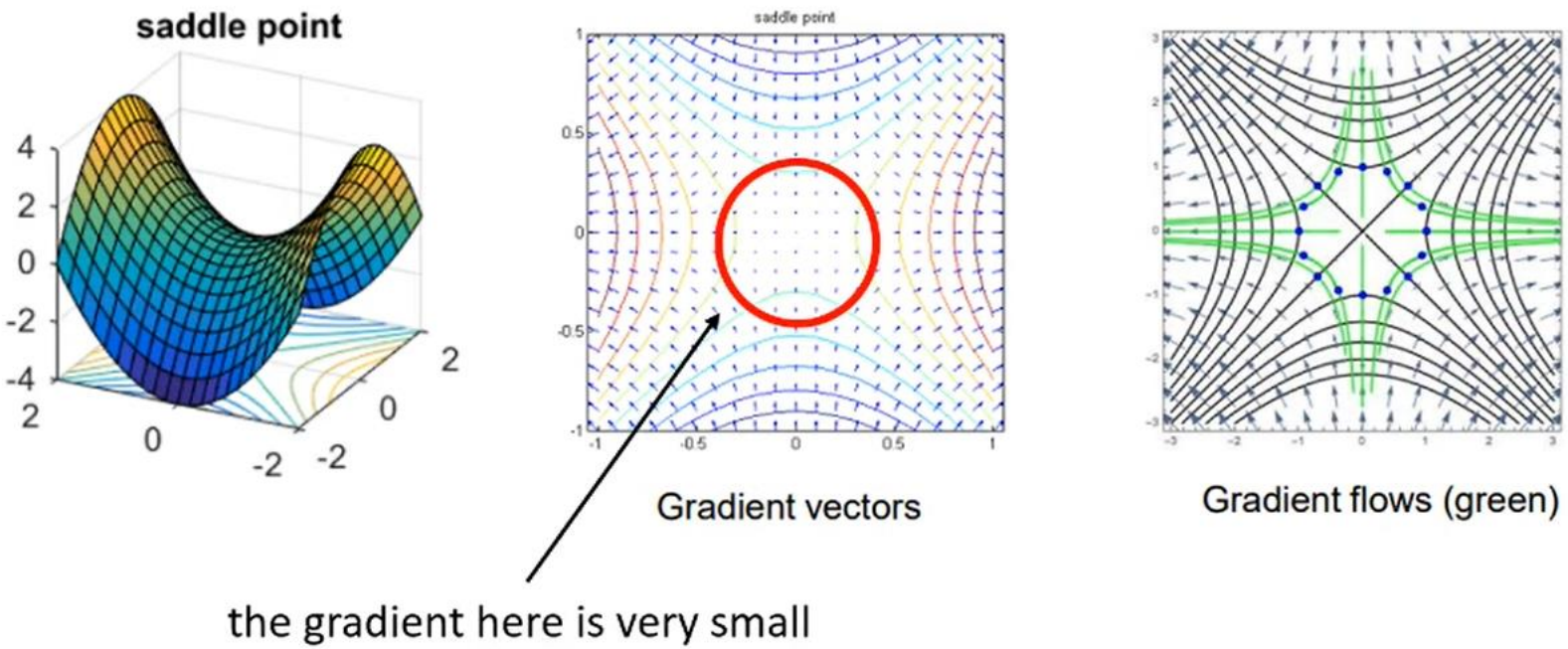
**Jason D. Lee**
University of Southern California
jasonlee@marshall.usc.edu

**Michael I. Jordan**
University of California, Berkeley
jordan@cs.berkeley.edu

**Barnabás Póczos**
Carnegie Mellon University
bapoczos@cs.cmu.edu

**Aarti Singh**
Carnegie Mellon University
aartisingh@cmu.edu

# The landscape of Neural Networks – saddle points

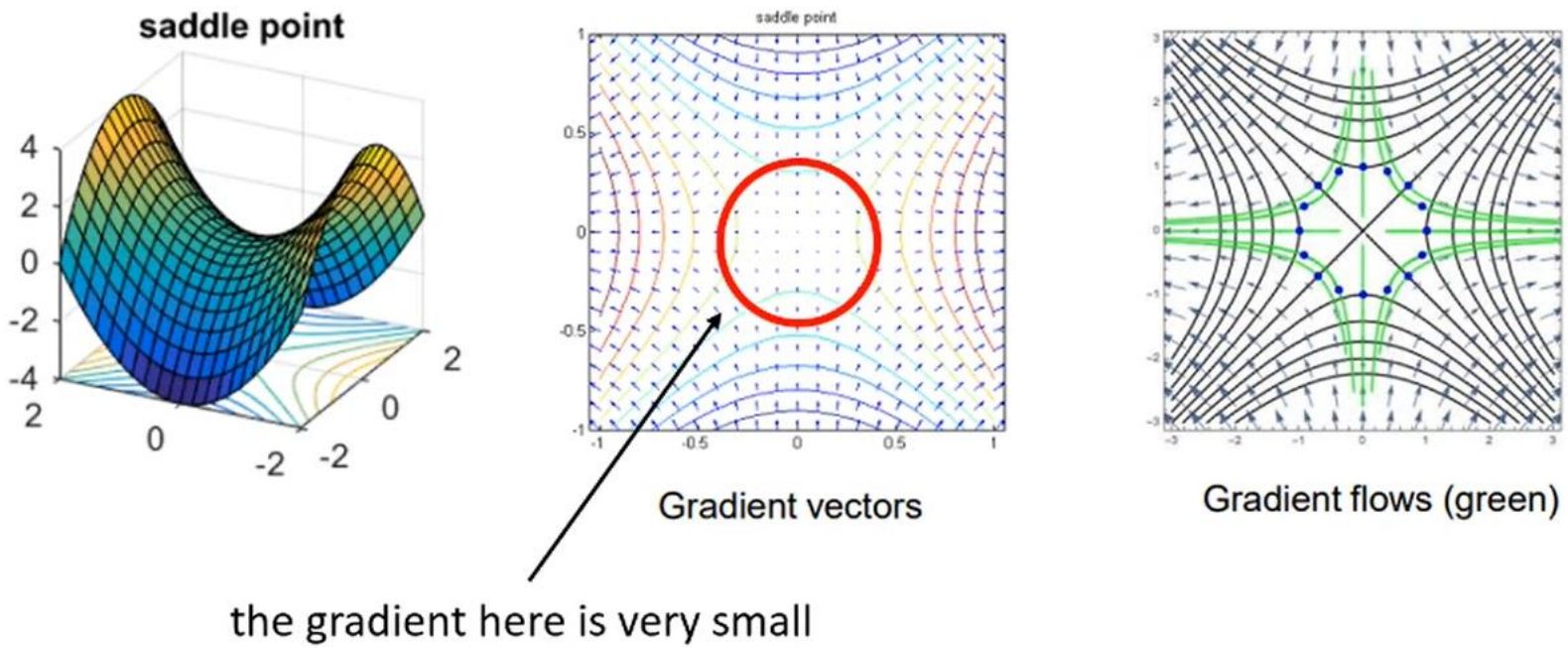



**Most stationary points in NNs are saddle points!**

**Escaping Saddle Points with Adaptive Gradient Methods**

Matthew Staib [1 2] Sashank Reddi [3] Satyen Kale [3] Sanjiv Kumar [3] Suvrit Sra [1]

# The landscape of Neural Networks – saddle points




**<u>Most stationary points in NNs are saddle points!</u>**

No clear way to deal with this, the stochasticity of the problem helps, as well as momentum and adaptive scaling.

*Deep Learning course by Sergey Levine CS 182 2021, Lecture 4: Optimization

# ANN training is optimising stochastic expensive functions

# ANN training is optimizing stochastic expensive functions

**Why is Gradient Descent expensive in ANNs?**

Assume a quadratic loss (not always the case):

$$loss(\boldsymbol{\theta}|\mathcal{D}_N) := \frac{1}{N}\sum_{i=1}^{N}\|\hat{\mathbf{y}}_i(\boldsymbol{\theta}) - \mathbf{y}_i\|_2^2$$

$N$ is extremely large in many Deep Learning applications, for example:

ImageNet has $1.5$ million images, and each image has a size of (on average and with pre-processing) $256 \times 256 \times 3$. This means $N \approx 294{,}912$**<u>million</u>**

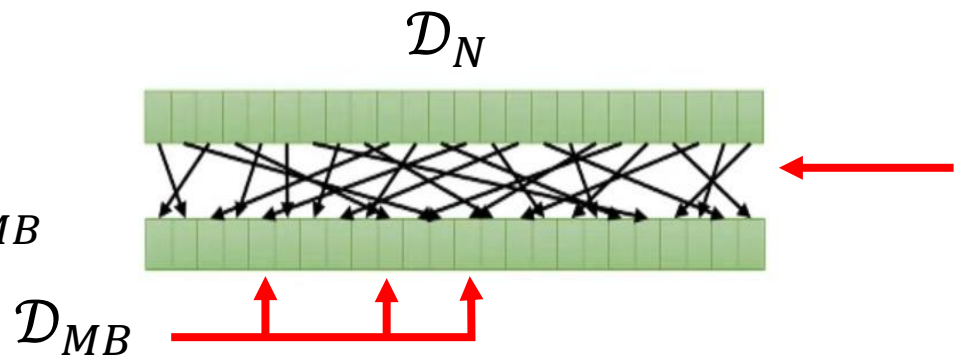
This is too expensive.

This is why people use '*minibatches*' in DL. That is, they take some randomised subset of datapoints $\mathcal{D}_{MB} \subset \mathcal{D}_N$

# ANN training is optimizing stochastic expensive functions

The steps to take now are:

1. Randomize $\mathcal{D}_N$ to create $\mathcal{D}_{MB}$
2. Sample $\mathcal{D}_{MB} \subset \mathcal{D}_N$




This is why it is called stochastic, as $\mathcal{D}_{MB}$ is a randomization.

3. Estimate stochastic gradient $\mathbf{g}_k \leftarrow -\nabla_{\boldsymbol{\theta}} \left( \frac{1}{MB} \sum_{i=1}^{MB} \|\hat{\mathbf{y}}_i(\boldsymbol{\theta}) - \mathbf{y}_i\|_2^2 \right)$
4. Take a stochastic gradient/Adam step $\boldsymbol{\theta}_{k+1} \leftarrow \boldsymbol{\theta}_k - \alpha \mathbf{g}_k$
5. Return to 1. until all the data has been 'seen'. This is called an *epoch*.

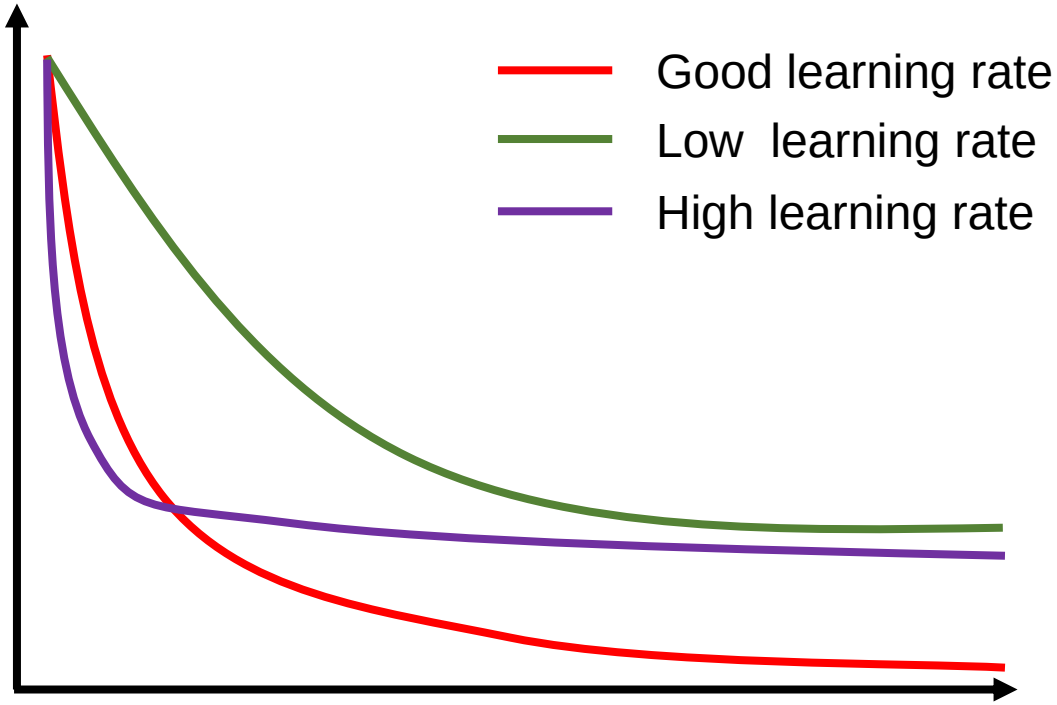
Notice each iteration is therefore expensive.

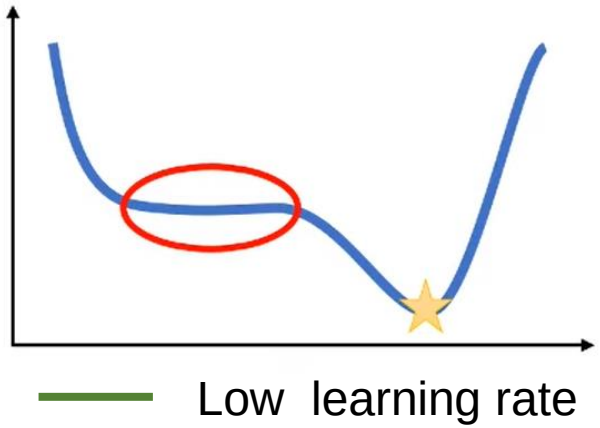
*Deep Learning course by Sergey Levine CS 182 2021, Lecture 4: Optimization

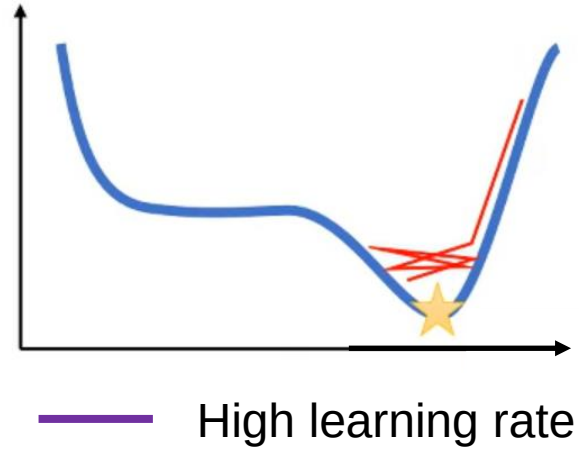
# ANN training is optimizing stochastic expensive functions

**Learning rates**

- Low learning rates result in bad results because they can get stuck in plateaus.
- High learning rates ‘bounce back’ and do not get to the minimums.




*Deep Learning course by Sergey Levine CS 182 2021, Lecture 4: Optimization

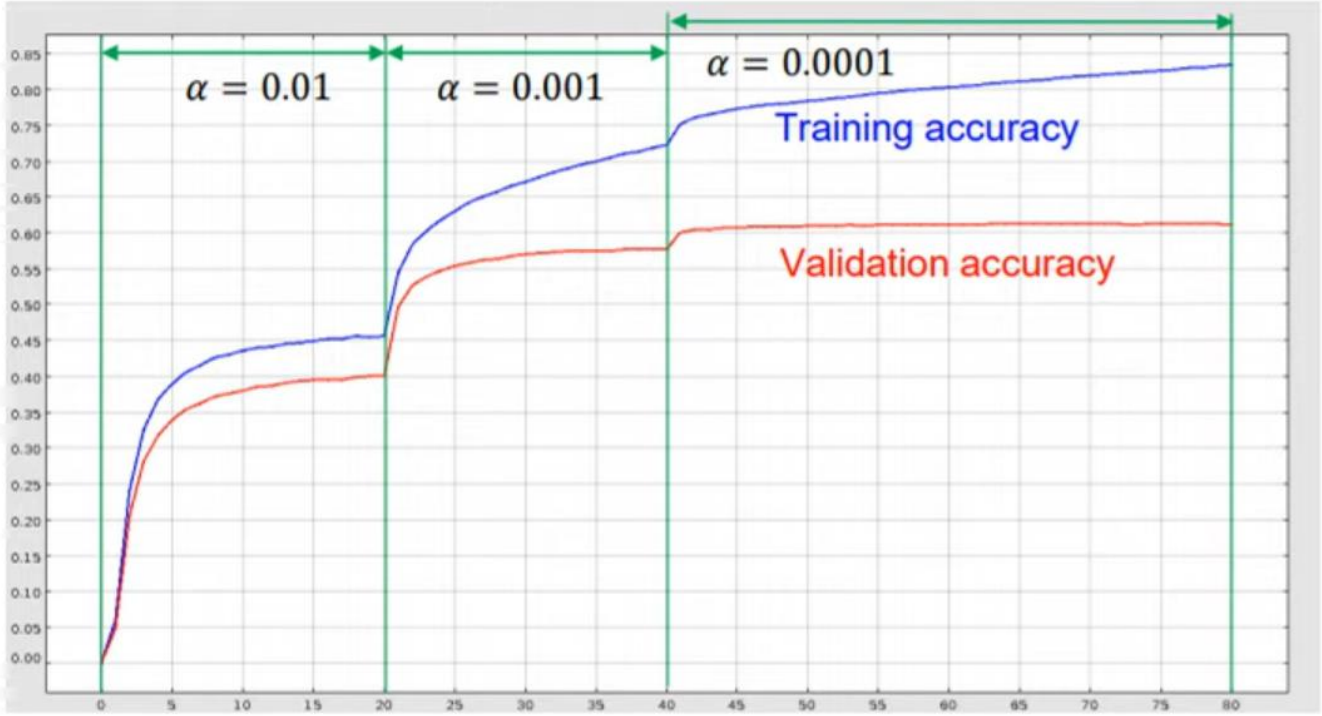
# Learning rate schedules

# ANN training is optimizing stochastic expensive functions

**Decaying learning rates**

- Learning rate decay schedules usually perform best SGD + momentum.
- Adam works well ad-hoc.

AlexNet trained on ImageNet



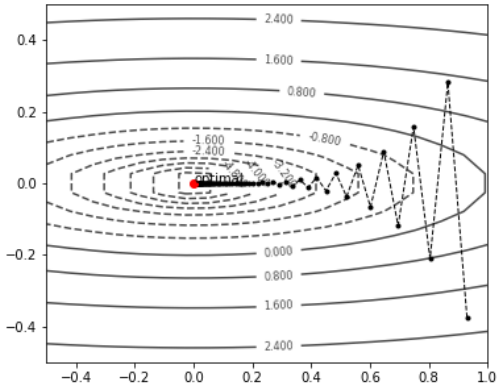

*Deep Learning course by Sergey Levine CS 182 2021, Lecture 4: Optimization

# Summary

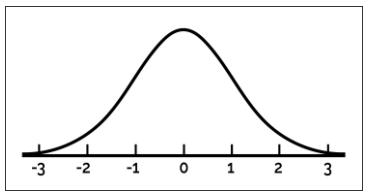
# Summary – Neural Networks

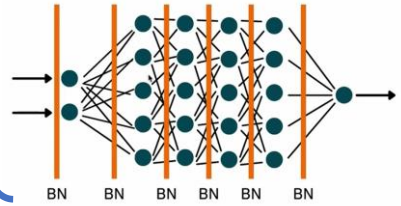
**Gradient Descent** – go-to algorithm for Deep Learning

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \ \nabla_x f(\mathbf{x}_k)$$

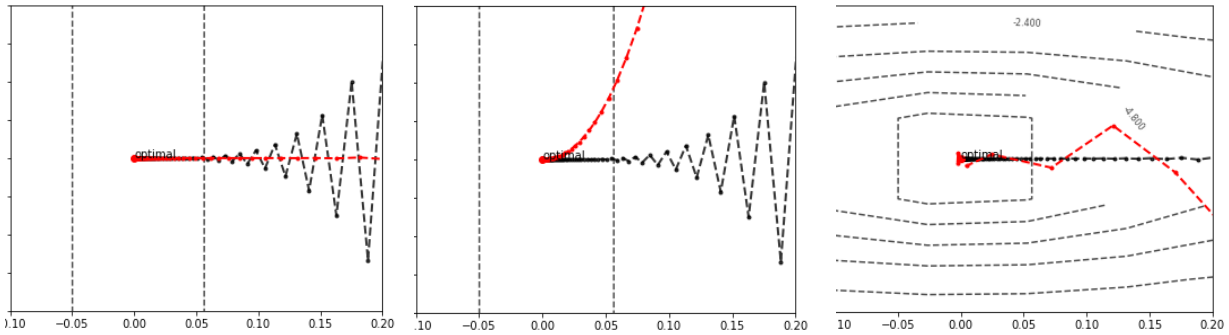



Training

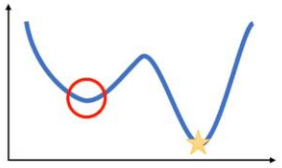




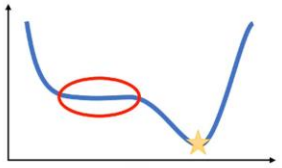

**Momentum** **RSMProp** **Adam**



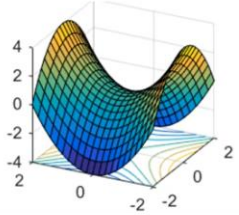

## Landscape of Neural Networks



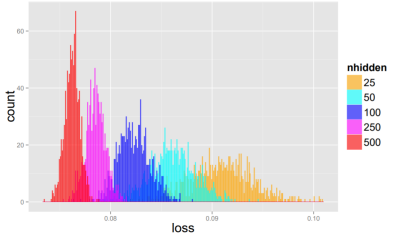
Local optimum

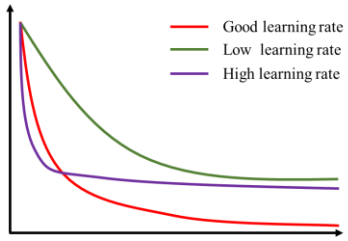


plateau



saddlepoint




Training ANNs in Engineering application:

- NNs are generally not as large.
- First order methods are typically used
- BFGS is a good algorithm for optimization.
- It's limited memory version is still used in DL:

L-BFGS

# Neural Networks – further resources

In this lecture we only covered the basics of Neural Networks. Some further resources:

- Specialization on **Deep Learning**: https://www.coursera.org/specializations/deep-learning
- NNs for **Images**: https://www.coursera.org/learn/convolutional-neural-networks
- NNs for **sequences**: https://www.coursera.org/learn/nlp-sequence-models
- **Book** on Deep Learning: https://www.deeplearningbook.org/

**Software**:

Most widely used NN software are PyTorch, JAX and Keras+Tensorflow

For consistency, in this course we will be using scikit-learn.

# Hands-on

Workbook on optimisation